

NEW-AGE GLOBAL UNIVERSITY FOR LIBERAL EDUCATION

Semester: II
Operating System
Category: Core
Course Code: CS1103
(Theory and Lab)

Course Outline

Unit – I

04 Hrs

Introduction to Operating Systems: Operating Systems –Definition Types- Functions -Abstract view of OS- System Structures – System Calls- Virtual Machines.

Unit – II

08 Hrs

Process Management: Process Concepts, Inter-process communication –Threads –Multithreading, Process Scheduling, First-Come, First-Served (FCFS), Shortest Job First, Round Robin (RR), Priority Scheduling.

Unit – III

06 Hrs

Synchronization and Deadlocks: Synchronization –Semaphores –Monitors, Hardware Synchronization –Deadlocks –Methods for Handling Deadlocks.

Unit – IV

12 Hrs

Memory Management: Memory Management Strategies –Contiguous and Non-Contiguous allocation –Paging –Virtual memory Management –File System Design and Implementation – Mass Storage Structure –Disk Scheduling and Management, FCFS, SSTF, and SCAN -I/O Systems- Overview of System Protection and Security.

PROCESS

A **Process** is a **program in execution**.

A **program** is a passive entity (stored on disk).

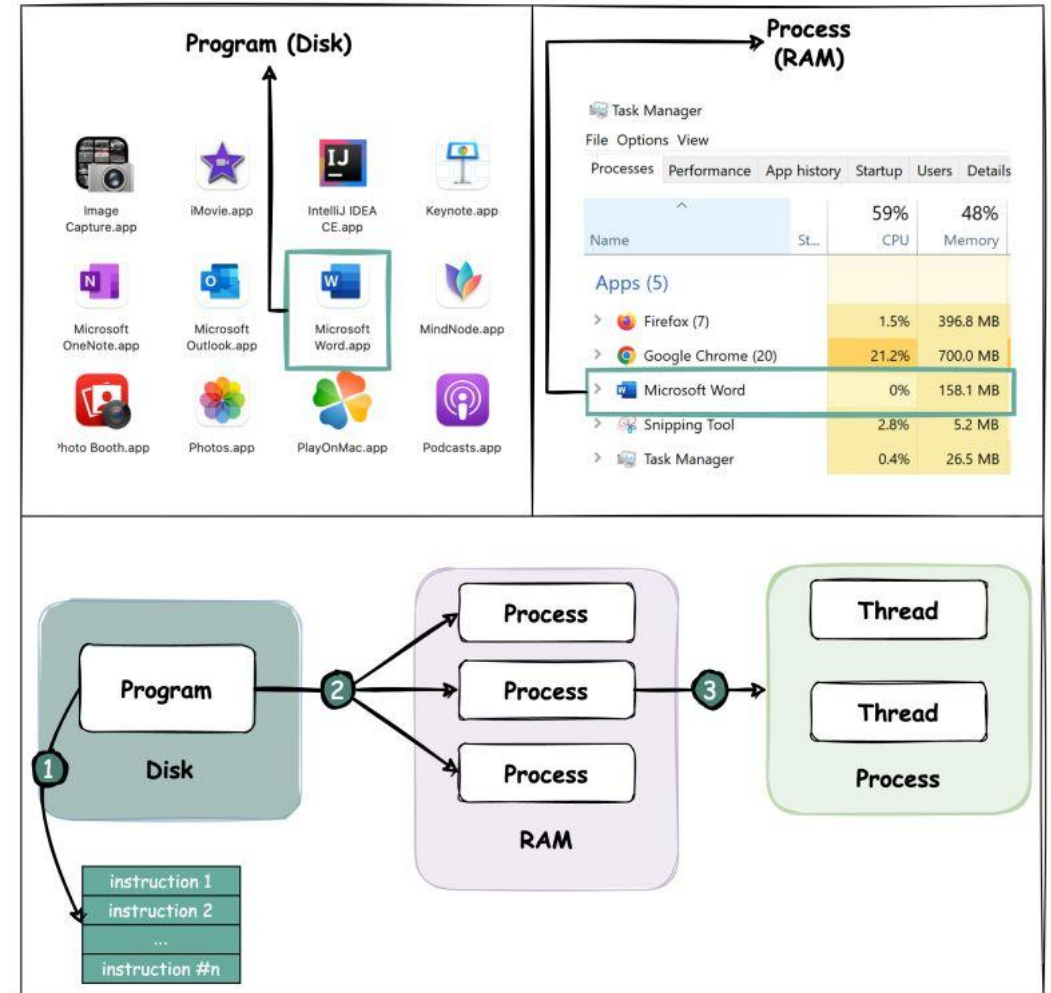
A **process** is an active entity (executing in memory).

Example:

When you double-click a browser, the program stored on disk becomes a **process** in memory.

Program vs Process vs Thread

blog.bytebytego.com



What is a Process?

A **Process** is a program in execution.

It is an active entity (unlike a program, which is passive).

A process includes:

- Program code
- Current activity (Program Counter)
- Stack (function calls, local variables)
- Heap (dynamic memory)
- Data section

Example: When you open Chrome, a process is created.

Components of a Process

Each process consists of the following components:

- ◆ **1. Text Section**

Contains the compiled program code.

- ◆ **2. Data Section**

Stores global and static variables.

- ◆ **3. Heap**

Used for dynamic memory allocation (malloc, new).

- ◆ **4. Stack**

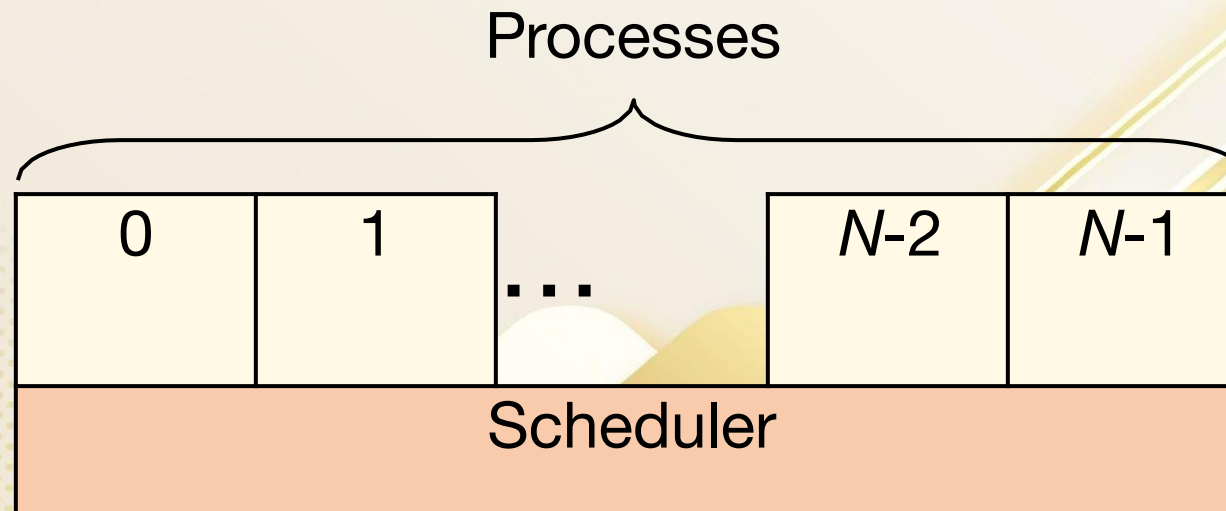
Stores function calls, local variables, return addresses.

- ◆ **5. Program Counter (PC)**

Indicates the address of the next instruction to execute.

Processes in the OS

- Two “layers” for processes
 - Lowest layer of process-structured OS handles interrupts, scheduling
 - Above that layer are sequential processes
Processes tracked in the *process table*
Each process has a process table entry



Process Operations

Process Operations

◆ Process Creation

A process can create another process (Parent–Child).

Example (UNIX): `fork()`

Example (Windows): `CreateProcess()`

◆ Process Termination

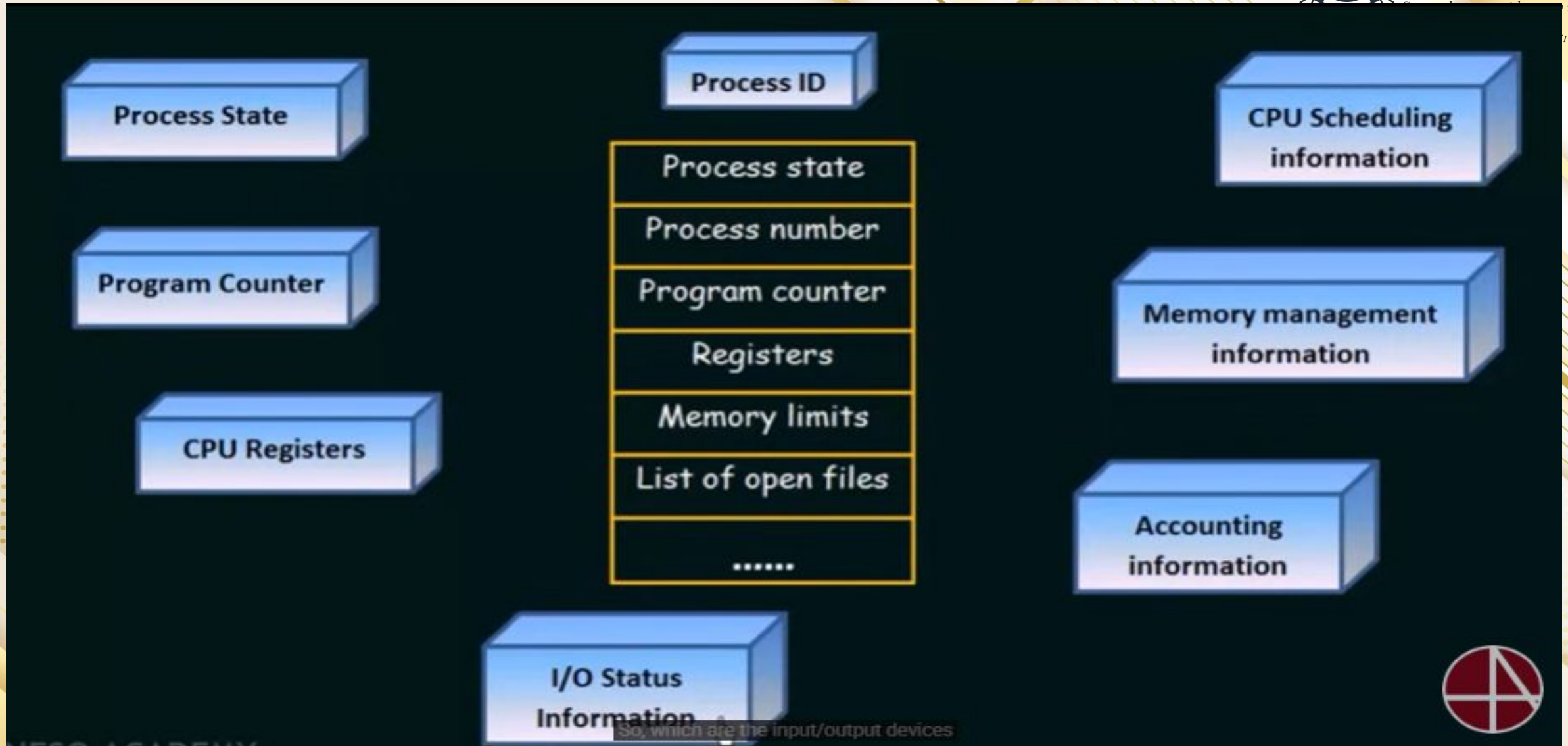
A process ends after execution.

Can terminate normally or abnormally.

Context Switching

- Switching CPU from one process to another.
- OS saves current process state in PCB.
- Loads the next process state.
- It causes overhead.

PROCESS TABLE



Structure of Process Table

◆ 1. Process Identification

Process ID (PID) – Unique number assigned to each process.

Parent Process ID (PPID) – Identifies the process that created it.

User ID – Identifies the owner of the process.

◆ 2. Process State

New – Process is being created.

Ready – Waiting for CPU allocation.

Running – Currently executing on CPU.

Waiting – Waiting for I/O or an event.

Terminated – Execution completed.

◆ 3. CPU Information

Program Counter – Address of next instruction to execute.

CPU Registers – Stores temporary execution data.

Priority – Determines scheduling preference.

Scheduling Information – Stores queue and scheduling details.

CONTD...

- ◆ **4. Memory Management Information**

Base & Limit Registers – Define memory boundaries of the process.

Page Table / Segment Table – Maps logical memory to physical memory.

Memory Allocation Details – Information about allocated memory.

- ◆ **5. I/O Status Information**

Open Files – List of files currently opened by the process.

Devices Allocated – Hardware devices assigned to the process.

I/O Requests – Pending input/output operations.

- ◆ **6. Accounting Information**

CPU Time Used – Total CPU time consumed.

Time Limits – Maximum allowed execution time.

Process Creation Time – Time at which process started.

Process Control Block

For each process there is a Process Control Block (PCB) which stores the process-specific information as shown below –

- Process State – The state of the process may be new, ready, running, waiting, and so on.
- Program counter – The counter indicates the address of the next instruction to be executed for this process.
- CPU registers -CPU registers are fast storage units that hold a process's execution state and must be saved with the program counter during interrupts to resume execution correctly.
- CPU scheduling information- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.



Process control Block

Process Control Block... contd.

- Memory-management information – This includes information such as the value of the base and limit registers, the page tables, or the segment tables.
- Accounting information – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- I/O status information – This information includes the list of I/O devices allocated to the process, a list of open files, and so on.



Process control Block

What is in the process table **entry**?

May be
stored on
stack



Process management

- Registers
- Program counter
- CPU status word
- Stack pointer
- Process state
- Priority / scheduling parameters
- Process ID
- Parent process ID
- Signals
- Process start time
- Total CPU usage

File management

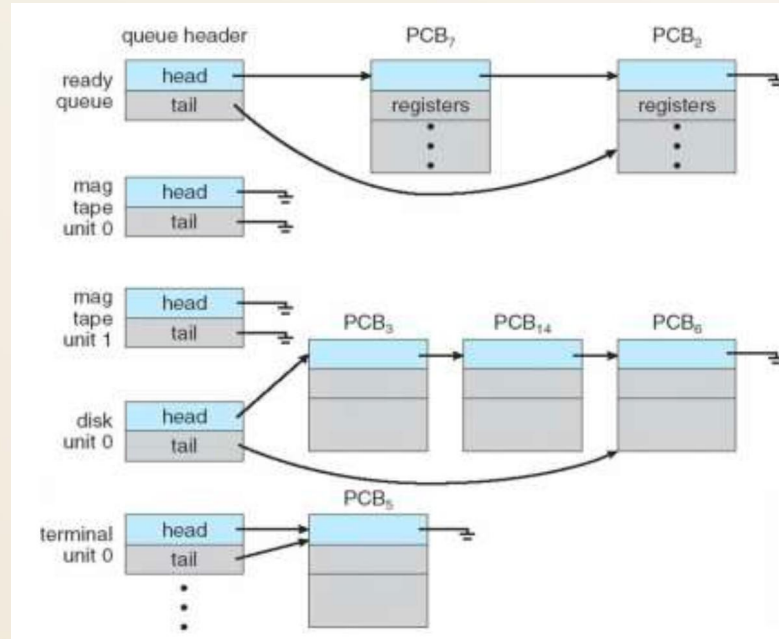
- Root directory
- Working (current) directory
- File descriptors
- User ID
- Group ID

Memory management

- Pointers to text, data, stack
or
Pointer to page table

Process Control Block... contd

The PCB simply serves as the repository for any **information** that may vary from process to **process**.



The ready queue and various I/O device queues

A new process is initially put in the ready queue. It waits in the ready queue until it is selected for execution and is given the CPU. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request, and then be placed in an I/O queue.
- The process could create a new subprocess and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue

Process **Control** Block... **contd**

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues

EXAMPLE:

In **Linux**:

The process table can be viewed using:

ps command

top command

The kernel maintains process information in a structure called task_struct.

In **Windows**:

Managed by the kernel object manager.

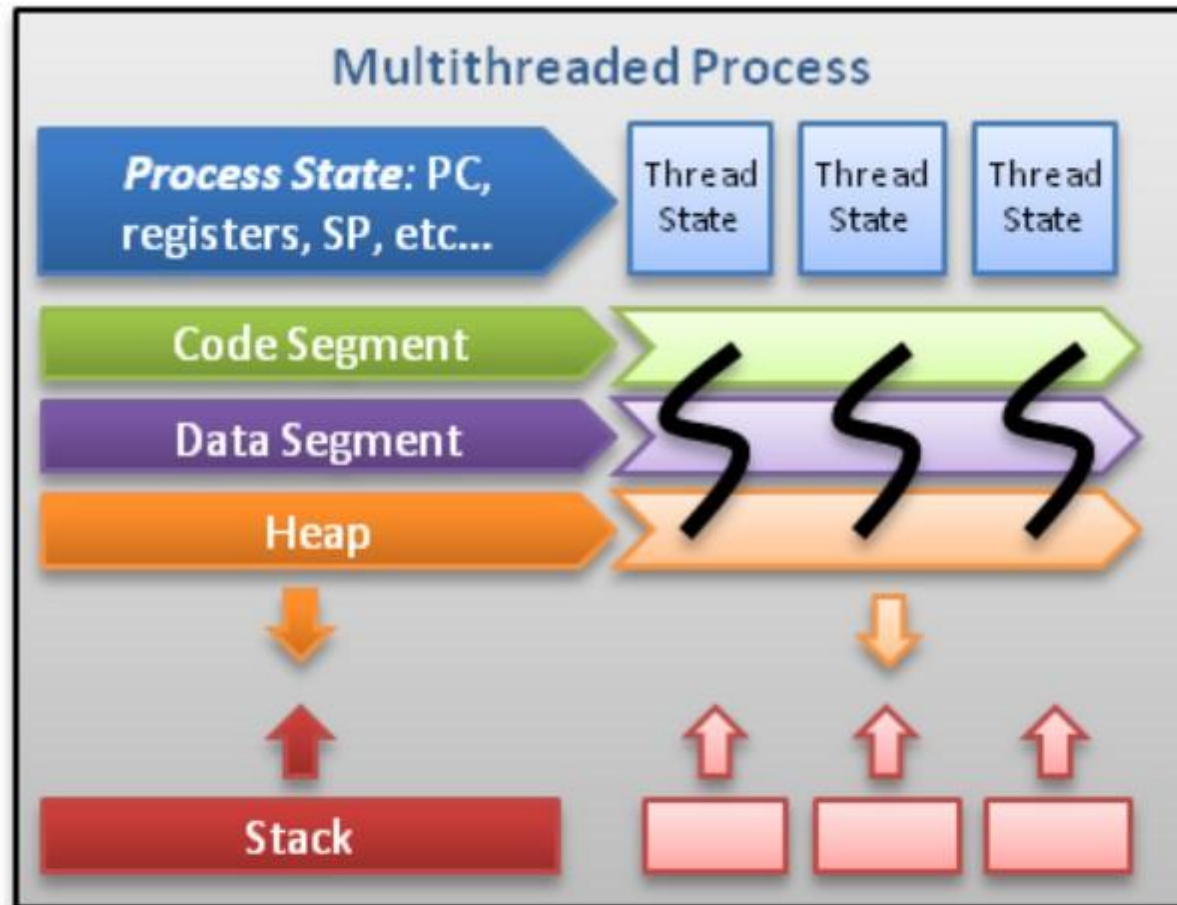
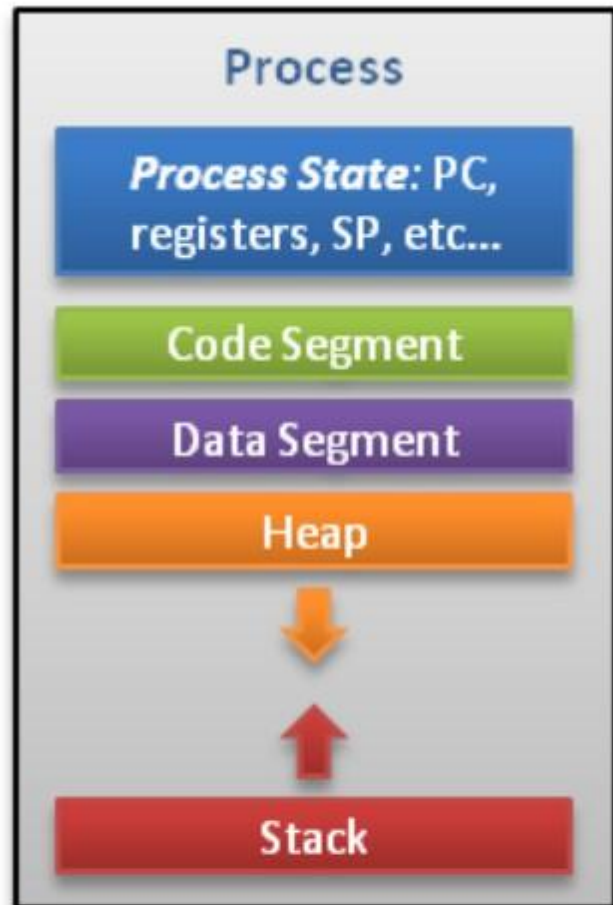
Can be viewed using Task Manager.

Q. What do think, happens in a **trap/** **interrupt?**

1. Hardware saves program counter (on stack or in a special register)
2. Hardware loads new PC, identifies interrupt
3. Assembly language routine saves registers
4. Assembly language routine sets up stack
5. Assembly language calls C to run service routine
6. Service routine calls scheduler
7. Scheduler selects a process to run next (might be the one interrupted...)
8. Assembly language routine loads PC & registers for the selected process

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Processes and Threads



Threads contain only necessary information, such as a stack (for local variables, function arguments, return values), a copy of the registers, program counter and any thread-specific data to allow them to be scheduled individually. Other data is shared within the process between all threads.

THREADS

A **Thread** is the smallest unit of CPU execution within a process.

Also called a **lightweight process**

Multiple threads can exist inside one process

Threads share:

- Code section
- Data section
- Files and resources

But each thread has its own:

- Program Counter
- Registers
- Stack

👉 Example: In a browser, one thread loads a webpage while another handles user input.

THREADS

Why Threads Are Needed?

- ✓ Improve responsiveness
 - ✓ Better CPU utilization
 - ✓ Faster context switching
 - ✓ Efficient resource sharing

Components of a Thread

Thread ID

Program Counter

Register Set

Stack

Thread Control Block (TCB)

Multithreading

What is Multithreading?

Multithreading is the ability of a process to execute multiple threads concurrently.

Multiple threads run within the same process simultaneously.

Advantages of Multithreading

- ✓ Increased responsiveness
- ✓ Resource sharing
- ✓ Economy (less memory usage)
- ✓ Scalability on multi-core processors

Types of Threads

Types of Threads

- ◆ **1. User-Level Threads**
 - Managed by user-level libraries
 - Faster switching
 - OS unaware of thread management
- ◆ **2. Kernel-Level Threads**
 - Managed by operating system kernel
 - Better parallelism
 - Slower than user-level threads

DIFFERENCES

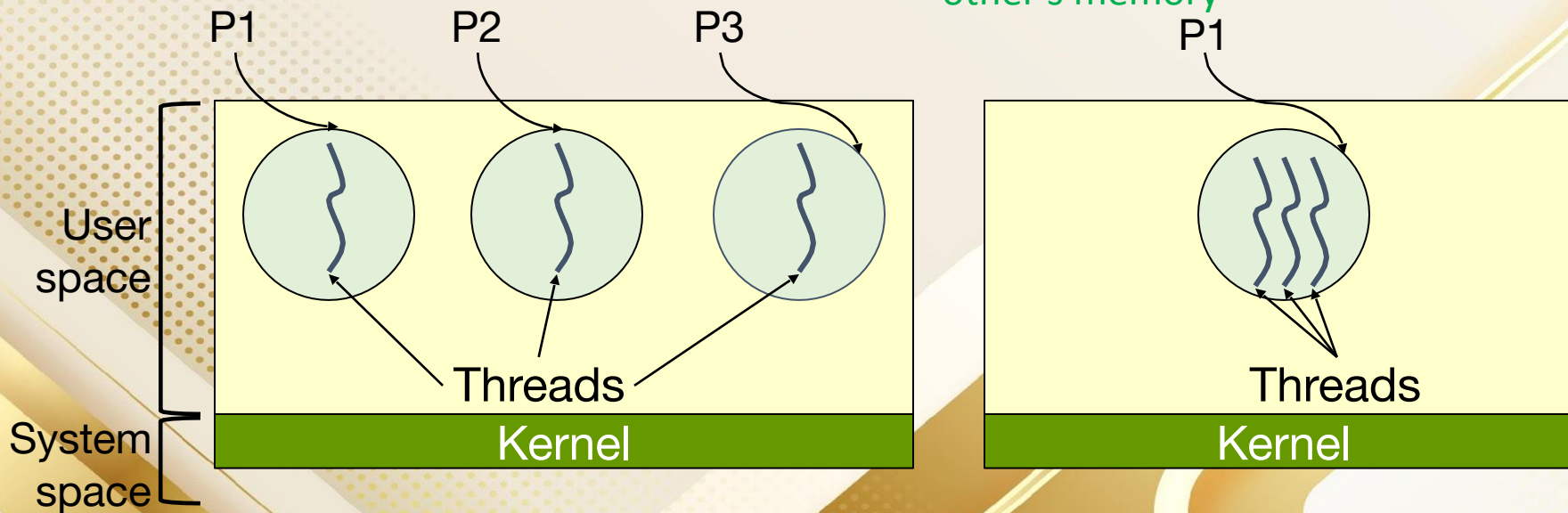
Feature	User-Level Threads	Kernel-Level Threads
Managed By	User library	Operating system
OS Awareness	No	Yes
Context Switch	Faster	Slower
Parallelism	Not possible	Possible
Blocking Call Effect	Blocks whole process	Blocks only that thread
Overhead	Low	High

- **User-Level Threads** are fast and lightweight but limited in parallelism.
- **Kernel-Level Threads** provide true concurrency but with higher overhead.

Threads: “processes” sharing **memory**

- Process (P) == address space
- Thread == program counter / stream of instructions
- Two examples are shown:
 - Three processes, each with one thread
 - One process with three threads

A thread within a process shares the same address space as the process itself, meaning all threads belonging to a single process can access the same memory locations without needing separate memory allocations for each thread; unlike processes which have their own distinct address spaces and cannot directly access each other's memory



Process & thread **information**

Per process items

Address space
Open files
Child processes
Signals & handlers
Accounting info
Global variables

Per thread items

Program counter
Registers
Stack & stack pointer
State

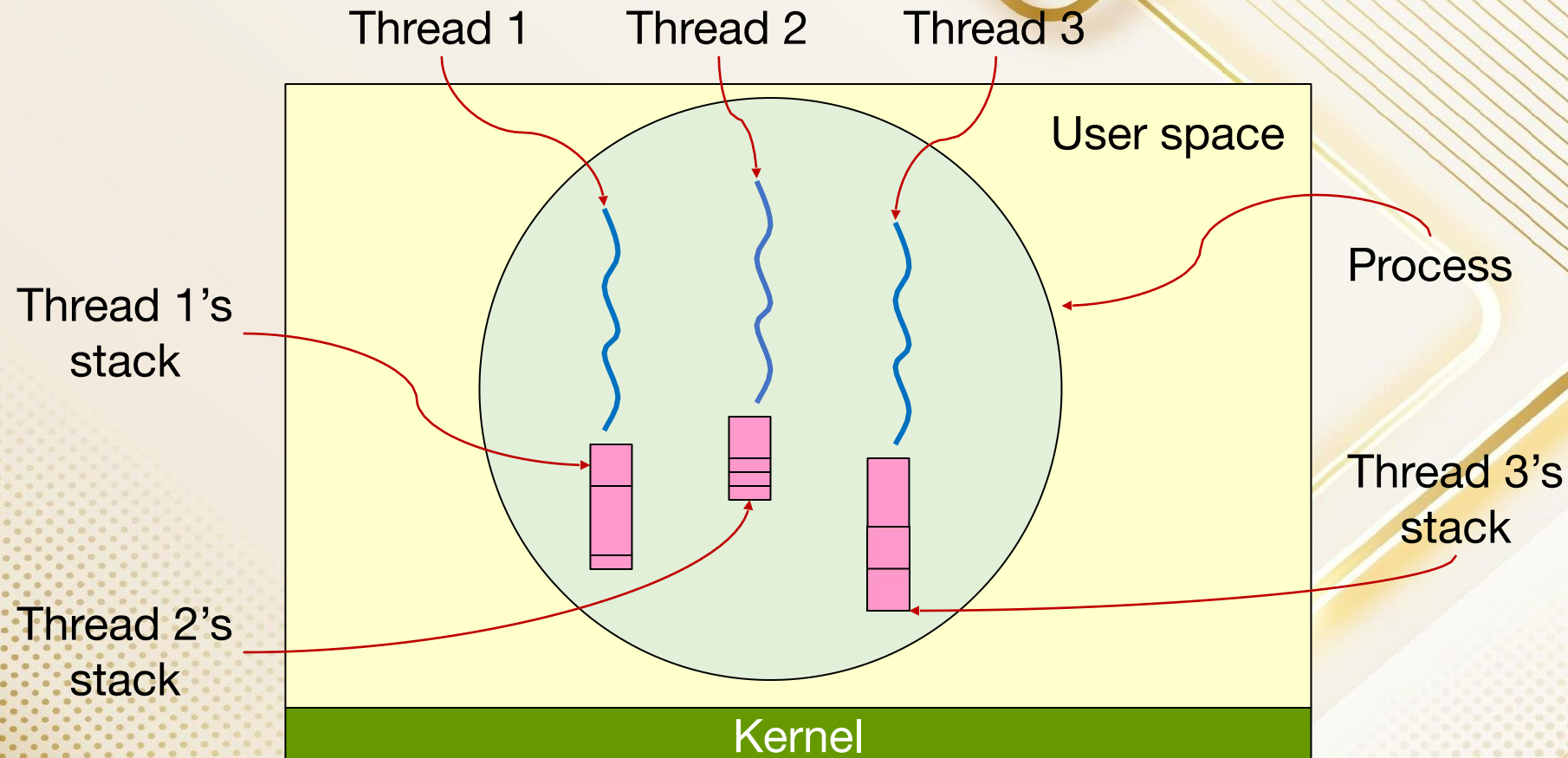
Per thread items

Program counter
Registers
Stack & stack pointer
State

Per thread items

Program counter
Registers
Stack & stack pointer
State

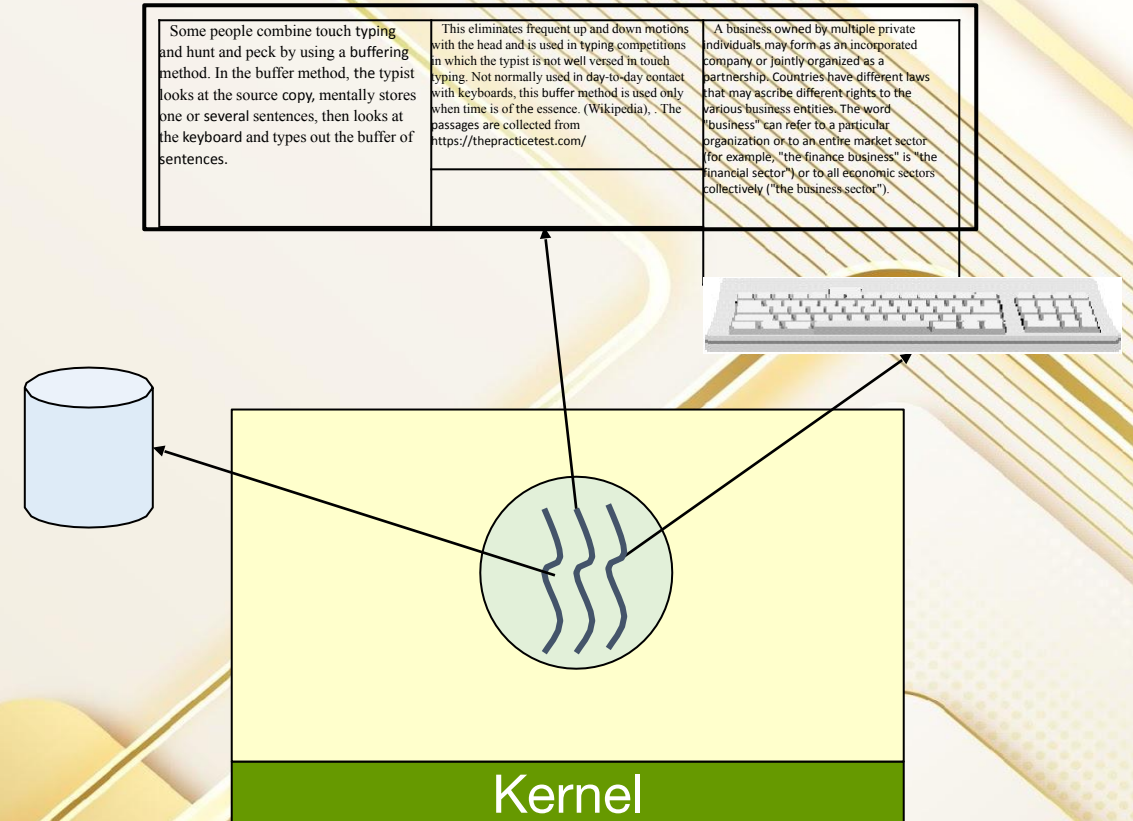
Threads & Stacks



=> Each thread has its own stack!

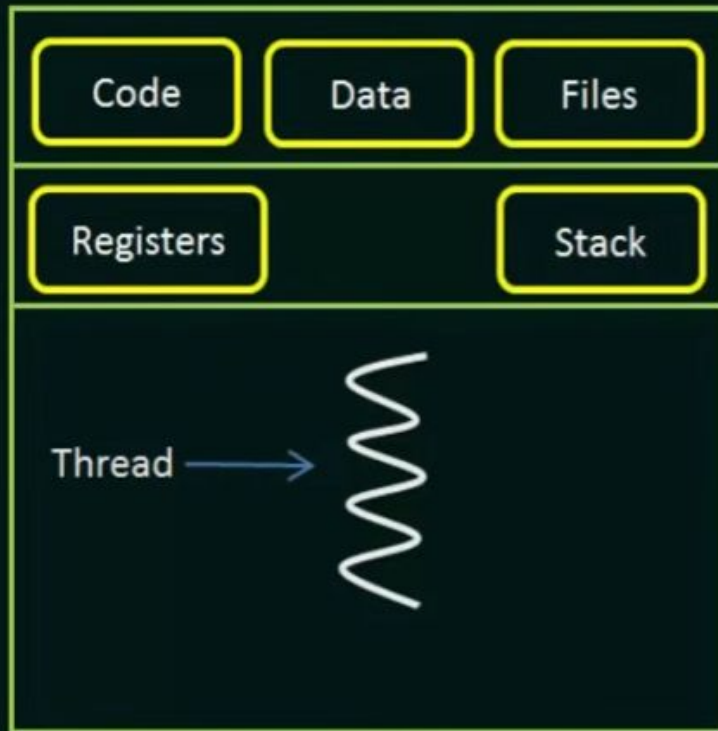
Q. What is the need for **Threads**?

- Achieve parallelism
- Allow a single application to do many things at once
 - Simpler programming model
 - Less waiting
- Threads are faster to create or destroy
 - No separate address space
- Overlap computation and I/O
 - Could be done without threads, but it's harder
- Example: word processor
 - Thread to read from keyboard
 - Thread to format document
 - Thread to write to disk

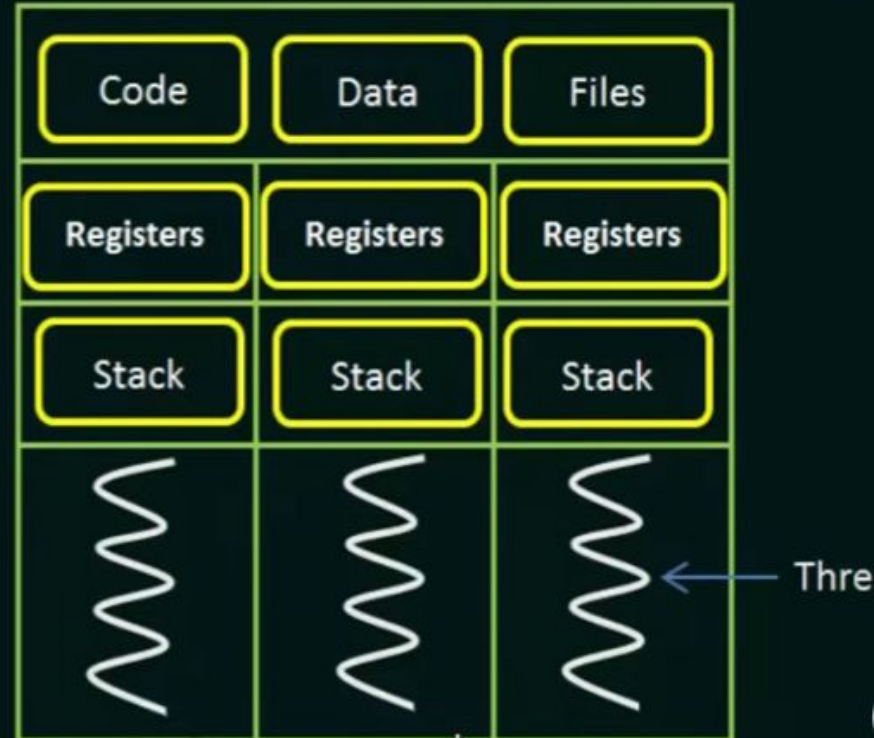


THREAD VS MULTITHREAD PROCESS

A traditional / heavyweight process has a **single thread** of control.
If a process has **multiple threads** of control, it can perform **more than one task at a time**.



Single-threaded process



Multi-threaded process

Explanation in terms of Threading **Perspective:**

This code is a typical structure for a producer-consumer model or a thread pool manager:

- **Single-Threaded:** If this loop runs in a single-threaded environment, all tasks will be handled sequentially, possibly causing delays.
- **Multi-Threaded:** If `handoffWork()` hands off the request to a worker thread (e.g., via a thread pool or task queue), this main thread acts as a dispatcher.
- **Concurrency:** This setup allows multiple worker threads to process requests in parallel, improving efficiency.


```
graph LR; Process[Process] --> Thread1[Thread]; Process --> Thread2[Thread];
```

The diagram illustrates a process spawning multiple threads. A central gray box labeled "Process" has two curved arrows pointing to two separate green boxes, each labeled "Thread".



Processes versus **Threads**

1. Processes are used when the tasks are essentially unrelated
2. Each process has its own address space and PCB
3. Address spaces are protected from each other
4. Switching between the process is done at the kernel level
5. Owned by one or more users

1. Threads are used when the tasks are actually part of the same job
2. Threads belonging to the same process share the process' address space, code data, and files , but not the registers and the stack.
3. No address space protections
Note: Among the threads of the same process.
4. Switching between the threads can be done either at the user level or the kernel level.
5. Usually owned by a single user.

Note: Threads belonging to a process

Layout of a process in **memory**

Explanation of Memory Layout:

•Text (Code) Segment:

- Contains the program's executable instructions (machine code).
- This segment is usually read-only to prevent accidental modification of the code.
- It's often shared among multiple processes running the same program.

•Data Segment:

- Stores global and static variables that are initialized before the program starts execution.
- Also includes the BSS (Block Started by Symbol) section, which holds uninitialized global and static variables. These are typically initialized to zero by the system.



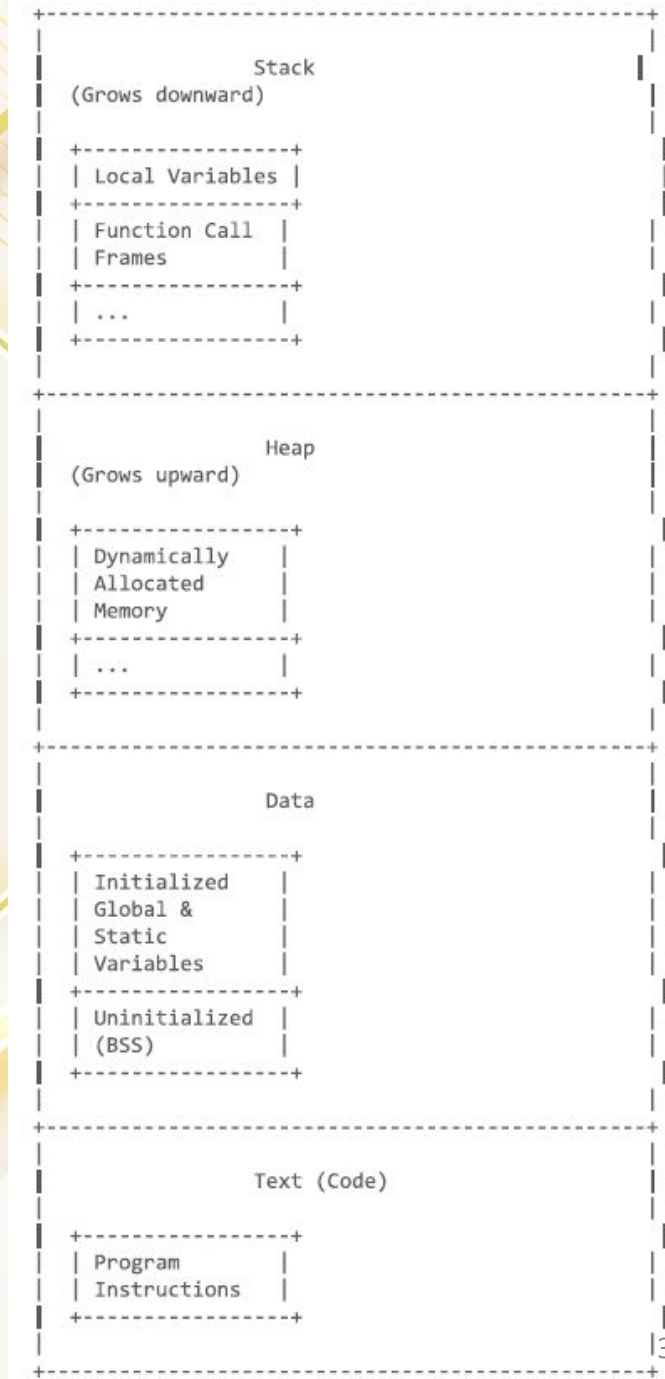
Layout of a process in memory

•Heap Segment:

- Used for dynamic memory allocation during program execution.
- Memory is allocated and deallocated using functions like malloc() and free() (in C/C++).
- The heap grows upward in memory.

•Stack Segment:

- Used for storing local variables, function call frames (including parameters and return addresses), and temporary values.
- The stack grows downward in memory.
- Each function call creates a new stack frame. When a function returns, its frame is popped off the stack.



Key concepts:

- **Virtual Address Space:** Each process has its own virtual address space, which is an illusion of having exclusive access to memory. The operating system translates these virtual addresses to physical addresses in RAM.
- **Memory Management:** The operating system is responsible for managing the allocation and deallocation of memory to processes.
- **Dynamic Memory Allocation:** The ability to allocate memory during program execution, as needed.
- **Stack Overflow:** Occurs when the stack grows beyond its allocated limit, often due to excessive recursion or too many local variables.
- **Memory Leak:** Occurs when dynamically allocated memory is no longer needed but is not freed, leading to a gradual depletion of available memory.

What is a **process**?

- ❑ Code, data, and stack
 - ❑ Usually (but not always) has its own address space
 - ❑ Program state
 - ❑ CPU registers
- ❑ Program counter (current location in the code)
- ❑ Stack pointer

Only one process can be running in the CPU at any given time!

Process **Scheduling**

Process types:

- **I/O bound Processes**
- **CPU bound Process**

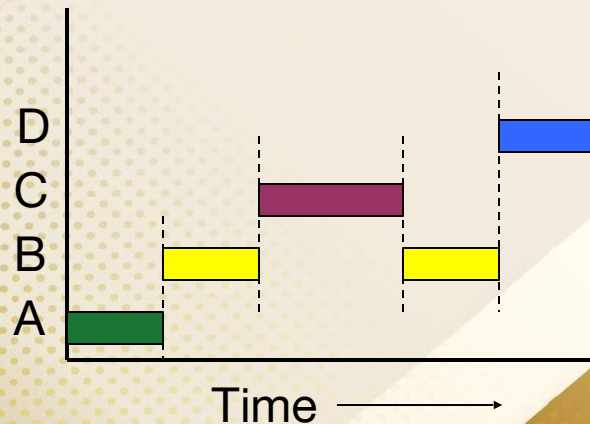
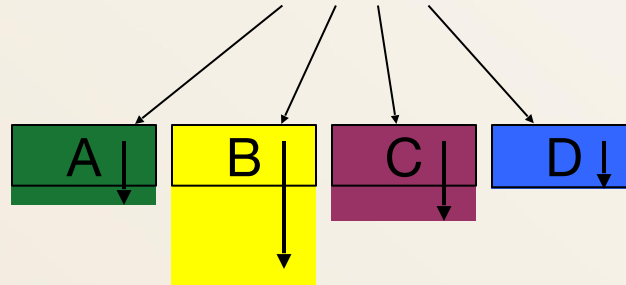
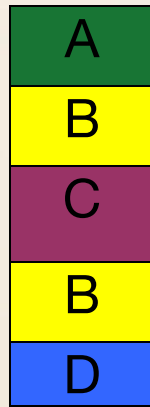
Things to Know:

- An **I/O-bound process** is one that spends more of its time doing I/O than it spends doing computations.
- A **CPU-bound process**, in contrast, generates I/O requests infrequently, using more of its time doing computations.

The process model

Single PC
(CPU's point of view)

Multiple PCs
(process point of view)



- ☐ Multiprogramming of four programs
- ☐ Conceptual model
 - ☐ 4 independent processes
 - ☐ Processes run sequentially
- ☐ Only one program active at any instant!
 - ☐ That instant can be very short...

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Two-State Process Model

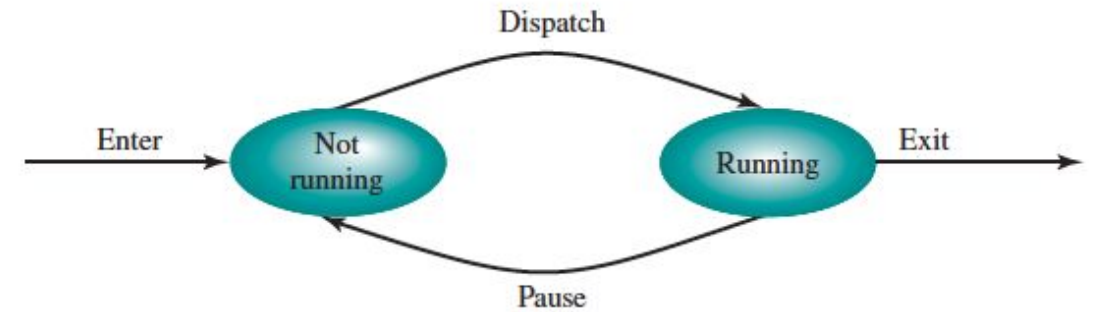
States in Two-State Model

1 Running

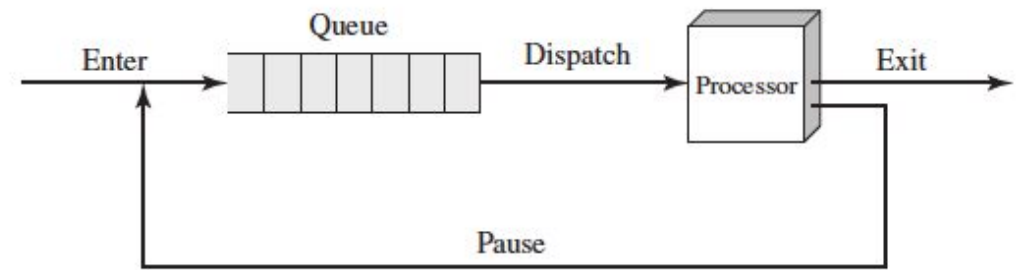
The process is currently executing on the CPU.

2 Not Running

The process is not executing.
It may be ready to run or waiting for I/O.



(a) State transition diagram



(b) Queueing diagram

Figure 3.5 Two-State Process Model

Three States Process Model

Three States

1 Running

Process is currently executing on the CPU.

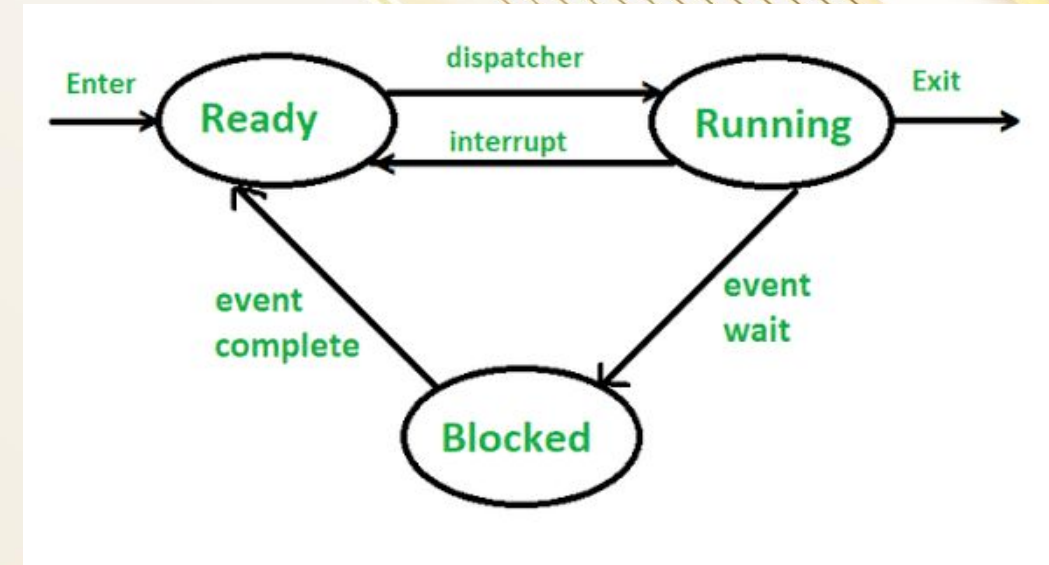
2 Ready

Process is ready to execute.

Waiting for CPU allocation.

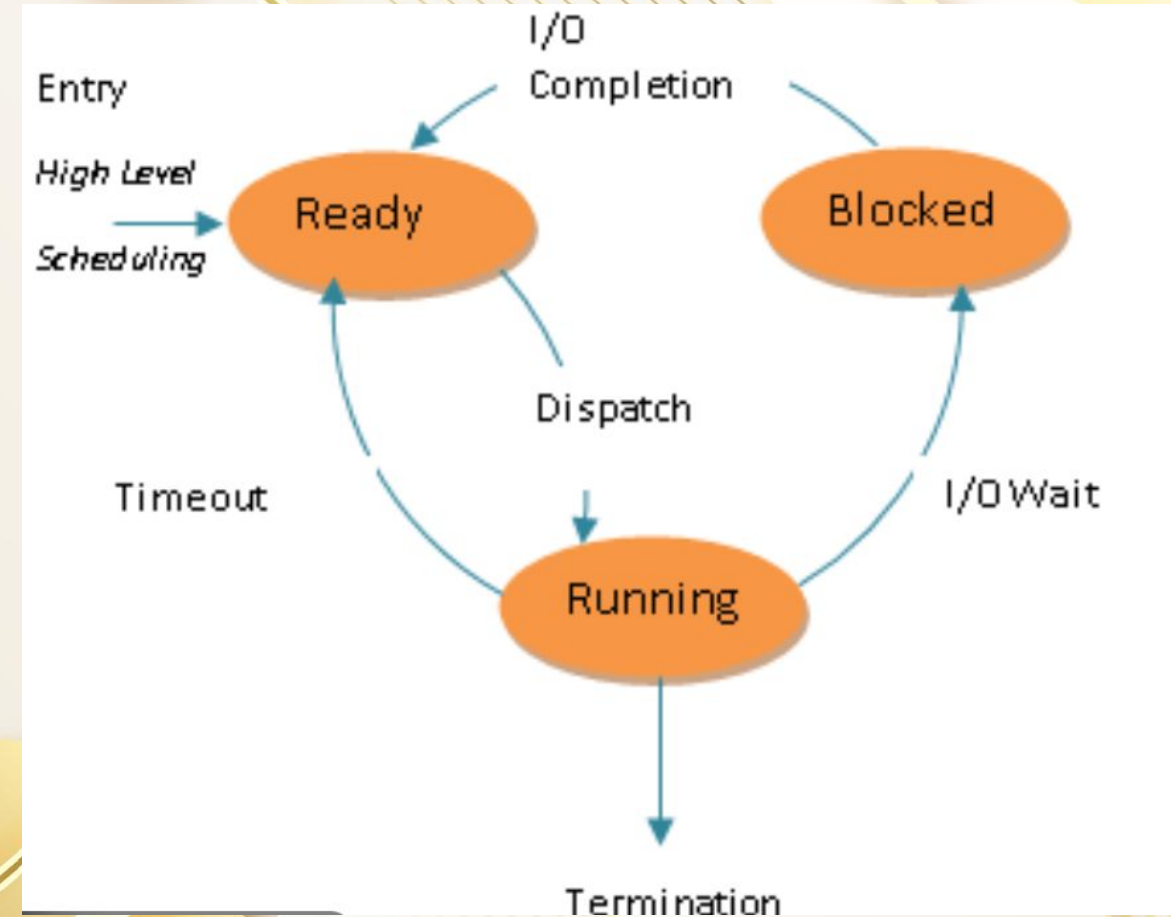
3 Waiting (Blocked)

Process is waiting for an event (e.g., I/O completion).

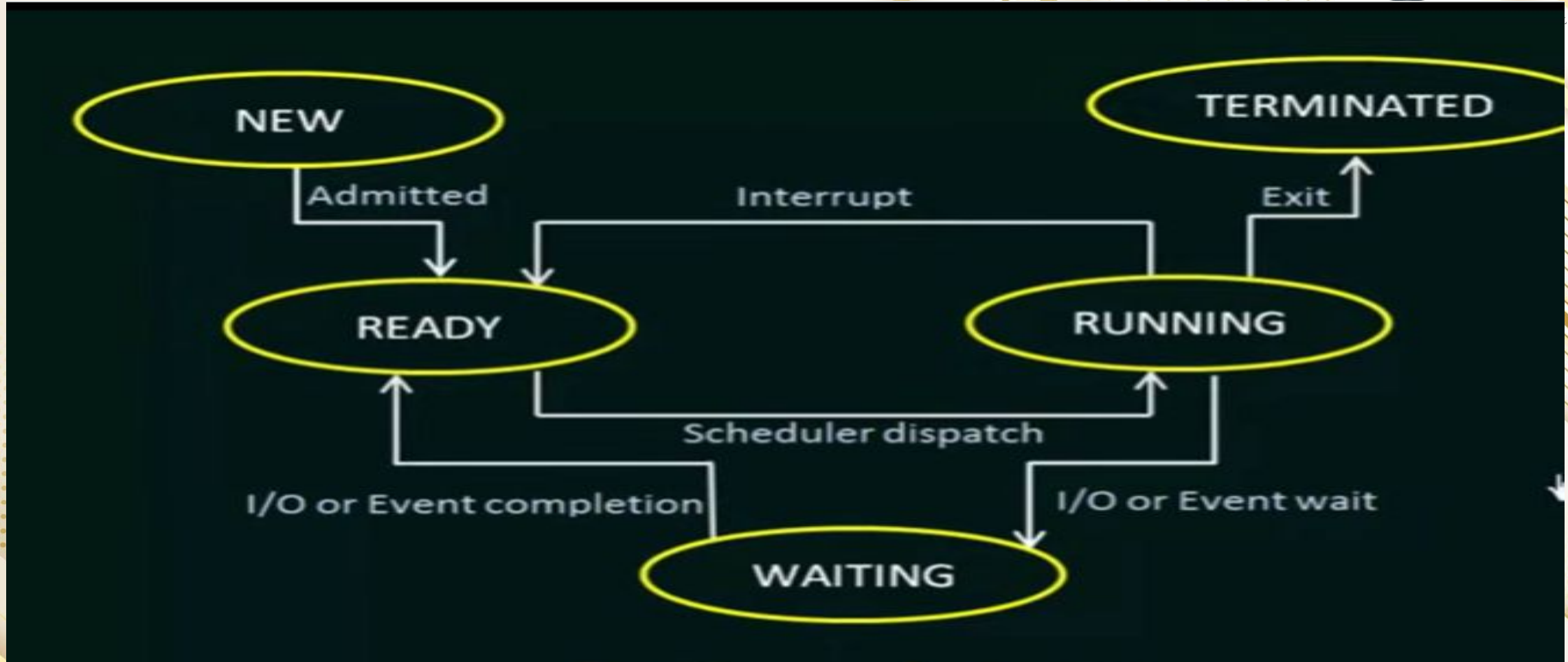


Three States Process Model

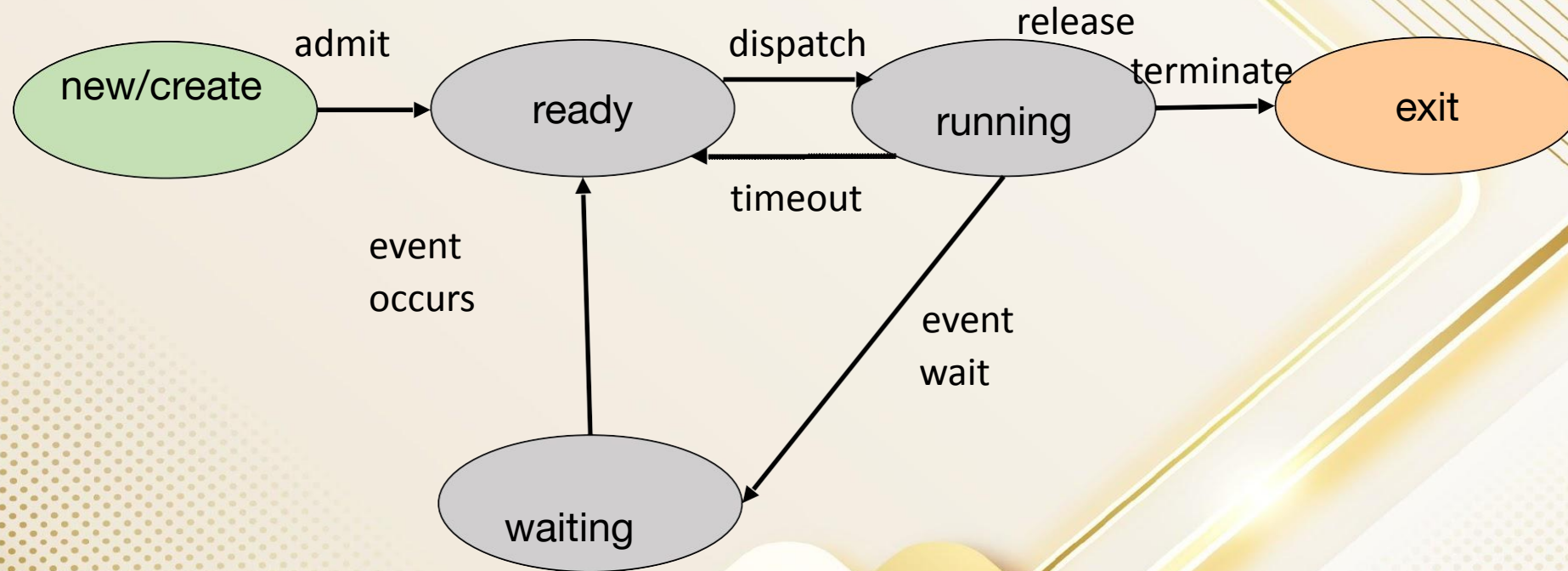
- **Ready → Running** : Process selected by CPU scheduler
- **Running → Ready** : Time slice expires (preemption)
- **Running → Waiting** : I/O request or event wait
- **Waiting → Ready** : I/O completed



FIVE STATE PROCESS MODEL



Process state Five-state Model: Definition of States



- When a new process is created, a new PCB entry is created by the OS, but still the resident set of the program is yet to be loaded into the memory, thus, OS cannot run it, so, the process is kept in “New” state.
- When the loading of the process onto the memory is completed, the process moves from “New” to “Ready” state.

Five-state **Model: Definition of States**

New

- The process is being created.
- OS is setting up the process control block (PCB) and allocating resources.

Ready

- The process is loaded into main memory.
- It is ready to run and waiting for CPU allocation.
- All required resources except the CPU are available.

Running

- The process is currently being executed by the CPU.
- At any time, only one process can be in the running state on a single CPU.

Five-state **Model: Definition of States**

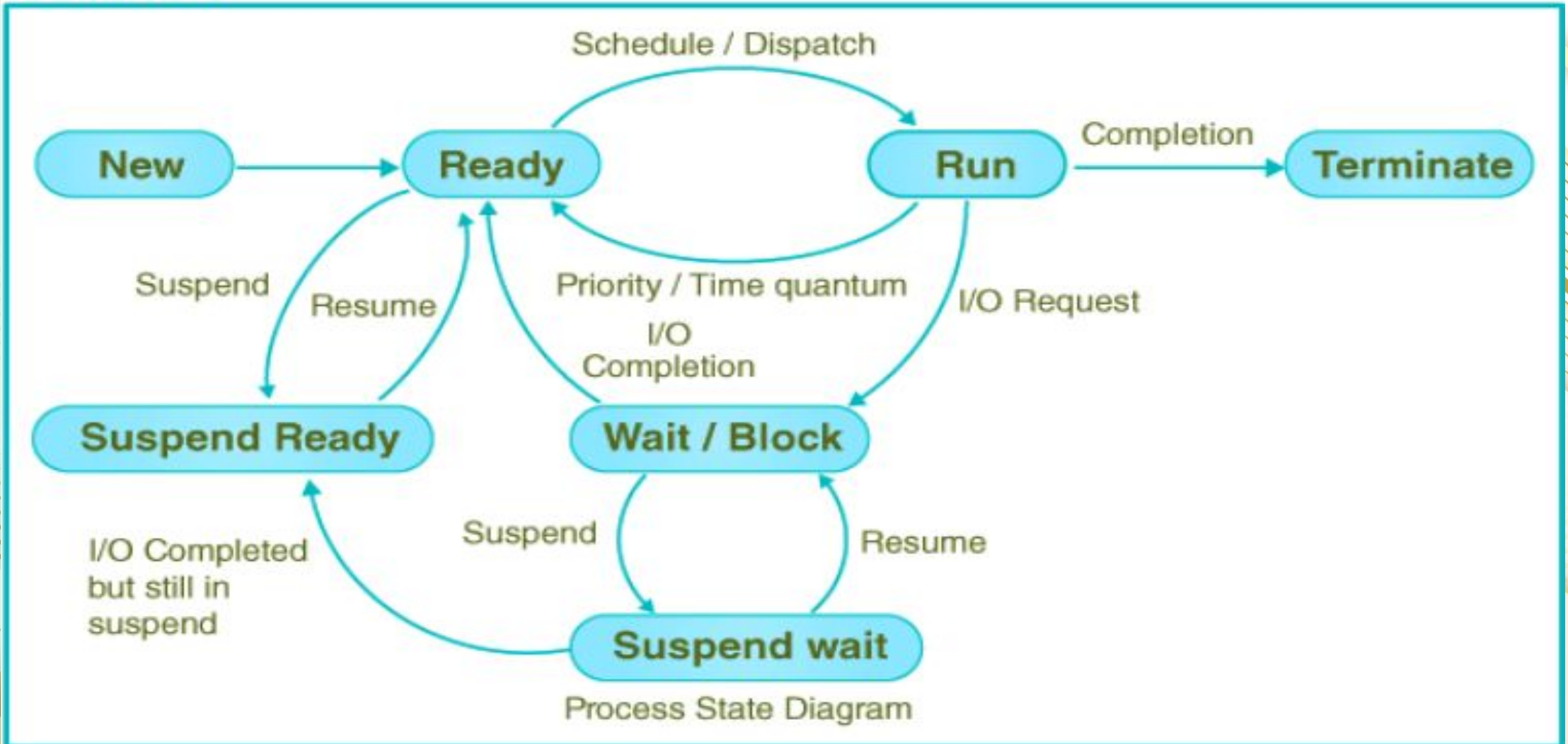
Waiting / Blocked

- The process cannot continue execution until some event occurs.
- Examples: waiting for I/O completion, user input, or a signal.

Terminated / Exit

- The process has finished execution.
- OS releases all resources held by the process.
- PCB is removed.

Five-state Model: Definition of States



Five-state **Model: Definition of States**

(Extended states in some OS models):

- **Suspended Ready**
 - The process is ready to run but has been swapped out of main memory.
 - Waiting to be brought back into memory.
- **Suspended Blocked**
 - The process is waiting for an event and is also swapped out of memory.
 - Must be moved to ready state after the event occurs and memory is available.
- If main memory is filled with blocked processes, the CPU may not have enough ready processes to execute.
- Lack of free memory can also prevent the OS from creating new processes or admitting more active processes.
- To keep the system efficient and responsive, main memory must be freed by moving some processes out of it

Five-state **Model: Definition of States**

Why Suspended State is Needed?

- Helps in **memory management**
- Supports **swapping**
- Improves **multiprogramming**
- Useful in handling large number of processes
- If main memory is filled with blocked processes, the CPU may not have enough ready processes to execute.
- Lack of free memory can also prevent the OS from creating new processes or admitting more active processes.
- To keep the system efficient and responsive, main memory must be freed by moving some processes out of it

Suspend state Reasoning

- OS chooses one of the Blocked processes residing in MM and change it to Suspend state, and moves the current process contents to disk.
- The space that is freed up in main memory can then be used to bring in some other processes or allow creation of new processes by the running processes.
- The suspended task is later brought into MM from the disk, directly into Ready state, when the event which it was blocked for, happens.

Note: Priority of the process plays a role in the OS choosing one of the Blocked process to Suspend state. The lowest priority process is chosen for moving to Suspend state and then to disk.

Key concepts:

- **Virtual Address Space:** Each process has its own virtual address space, which is an illusion of having exclusive access to memory. The operating system translates these virtual addresses to physical addresses in RAM.
- **Memory Management:** The operating system is responsible for managing the allocation and deallocation of memory to processes.
- **Dynamic Memory Allocation:** The ability to allocate memory during program execution, as needed.
- **Stack Overflow:** Occurs when the stack grows beyond its allocated limit, often due to excessive recursion or too many local variables.
- **Memory Leak:** Occurs when dynamically allocated memory is no longer needed but is not freed, leading to a gradual depletion of available memory.

A Sequential Program with I/O Wait

- Here, the program is waiting for a Char (from serial port) to be received, to proceed further.
- The main program would loop here until SerFlag is set indicating the arrival of a char, by the Serial ISR.

Loop in the main program

```
Loop: Load R7, SerFlag  
      CMP R7, 0  
      JMPEQ Loop
```

Note: R7 is loaded with the value of a **SerFlag variable** which is set to One when a Char is received on the serial port inside the Serial ISR

- With a single CPU, if this program is running, there is no useful job done by the CPU until a Char is received.
- This wastes the precious CPU cycles and power.

Interprocess Communication

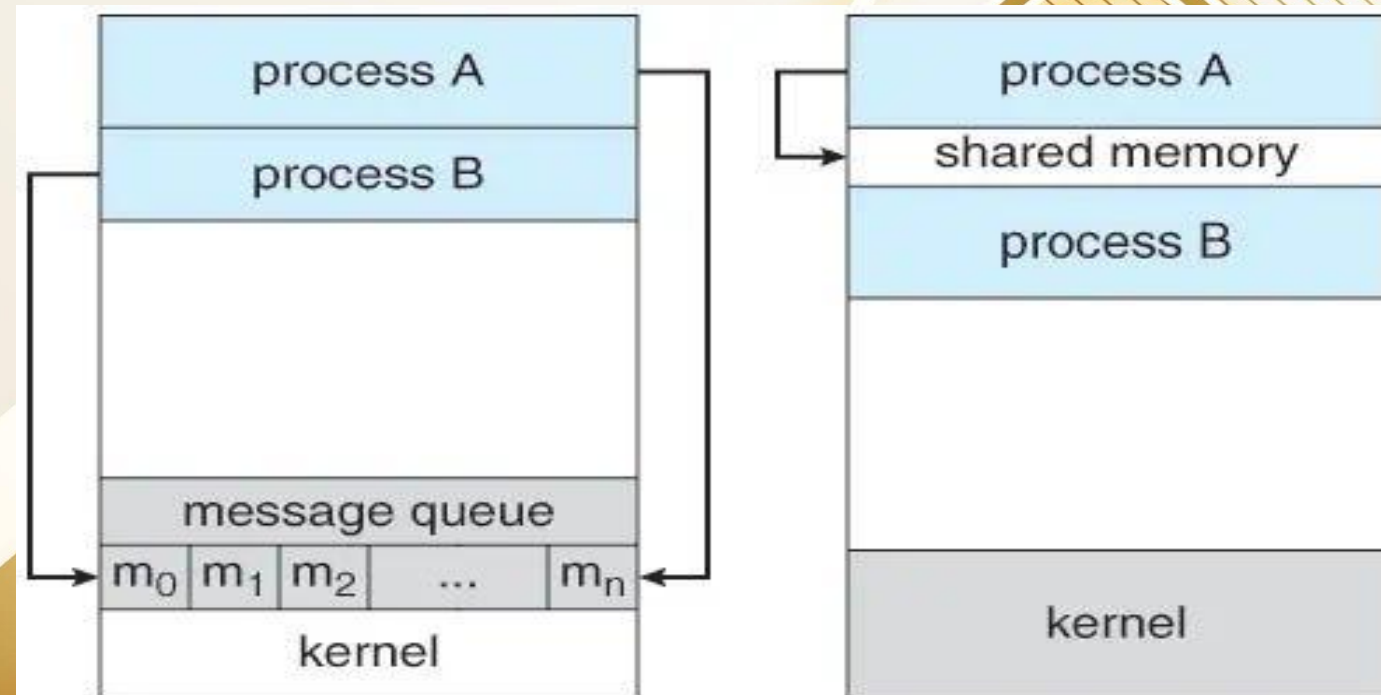
- **Interprocess Communication (IPC)** is a mechanism that allows processes to **communicate and synchronize** with each other in an Operating System.
- When processes are **cooperating**, they must exchange data safely and efficiently — this is done using IPC.

Why??

- **Information sharing** – Multiple processes access common data
- **Computation speedup** – Parallel task execution
- **Modularity** – System divided into smaller processes
- **Convenience** – Perform multiple tasks simultaneously

Interprocess Communication

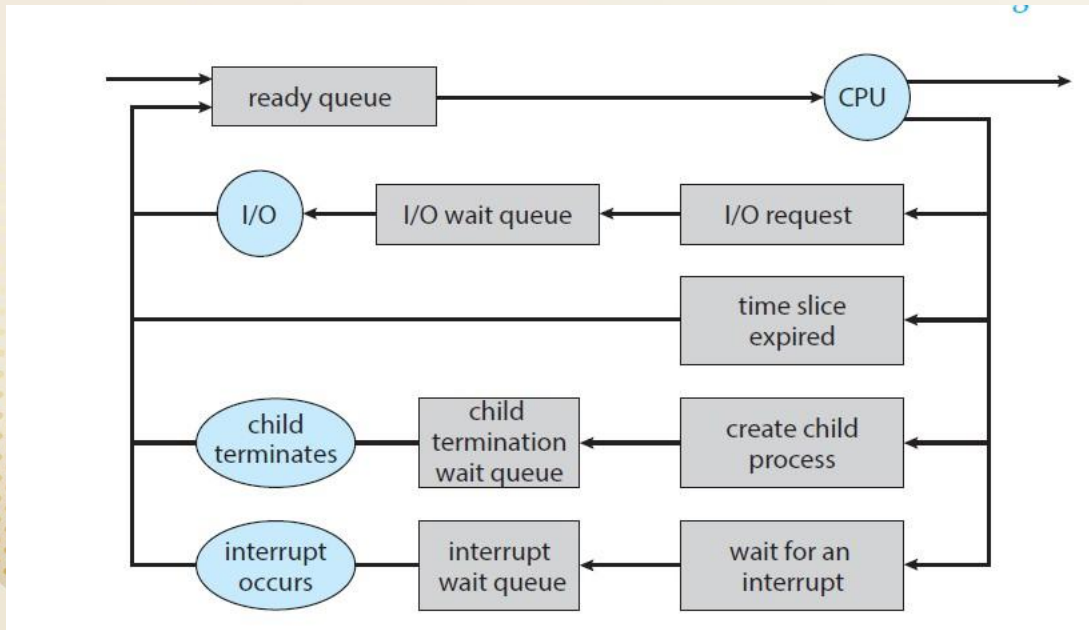
- Processes within a system may be independent or cooperating
- Cooperating process can affect or be affected by other processes, including sharing data
- Cooperating processes need interprocess communication (IPC)
- Two models of IPC
 - Shared memory
 - Message passing



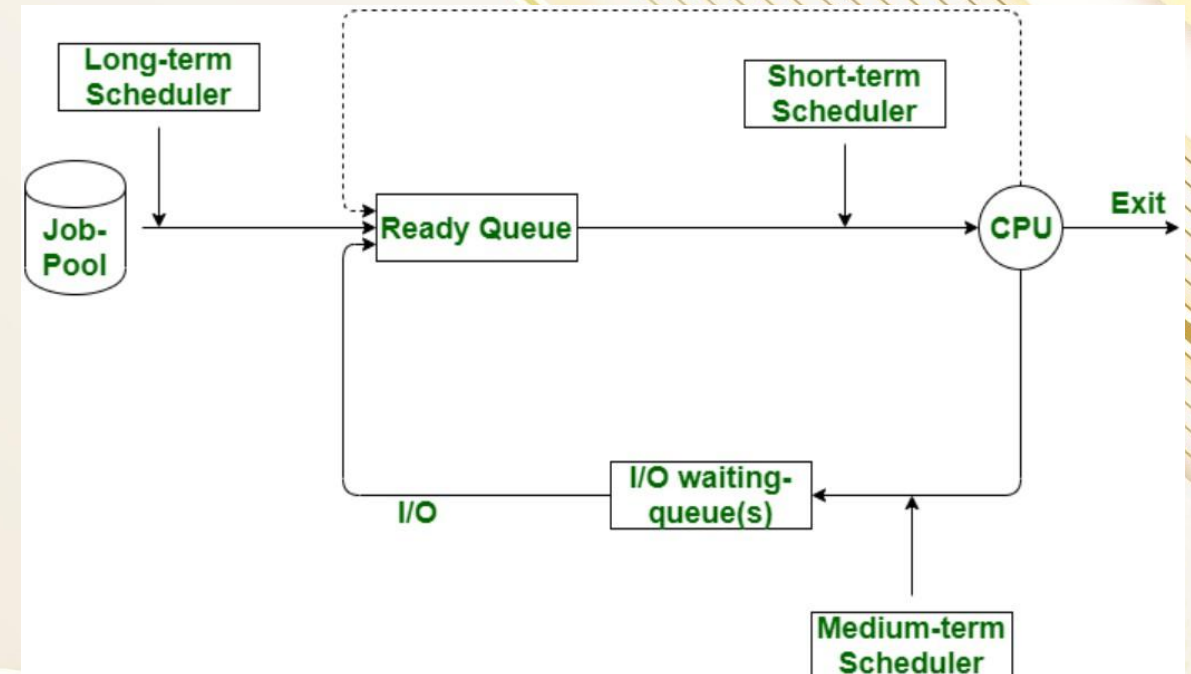
(a) Message Passing

(b) Shared Memory

Scheduling Queues and schedulers

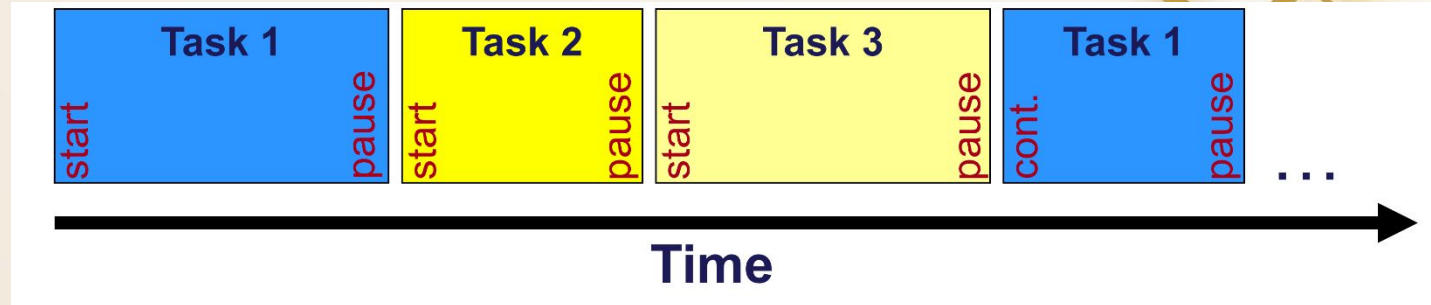


Queueing-diagram representation of process scheduling



Schedulers: Long, medium and short term

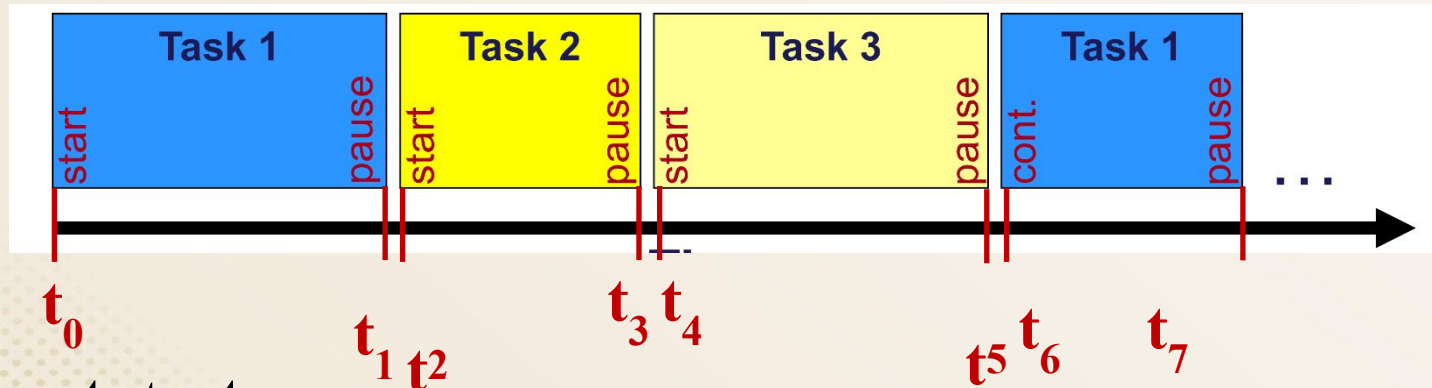
What does a Scheduler do?



- The scheduler decides when to stop the currently executing process and selects the next process to run on the CPU.
- When a process is removed from the CPU, its **context/status** is saved in the **Process Control Block (PCB)**.
- The context of the next selected process is then loaded, allowing it to execute on the CPU.

Quiz 1: When does the Scheduler run?

● If the timings are given as shown below, choose the option which shows all the time slots when the scheduler runs.



- A. From t_0 to t_1
- B. From t_0 to t_7
- C. From t_1 to t_2
- D. None of the above

ANS: D

The scheduler gets the control in regular interval or when a currently running process relinquishes the CPU voluntarily and picks the next process to run on the CPU.

Here, the scheduler runs during the time slots: t_1 to t_2 , t_3 to t_4 , t_5 to t_6 , ...

Scheduling

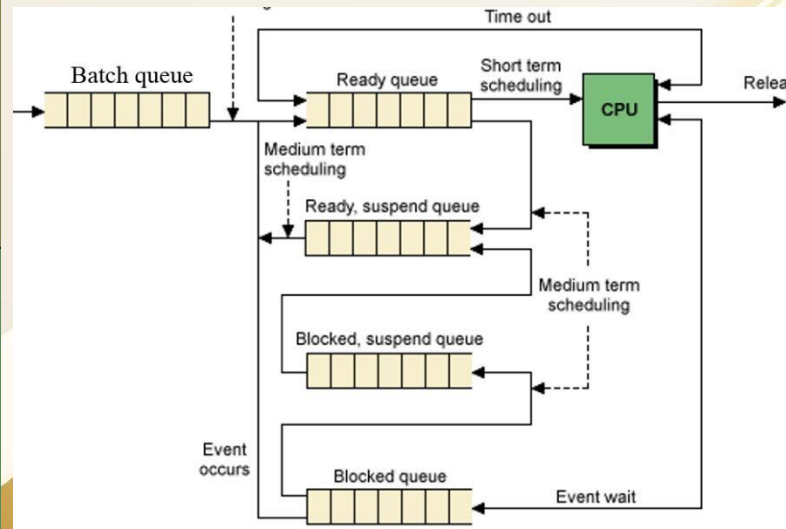
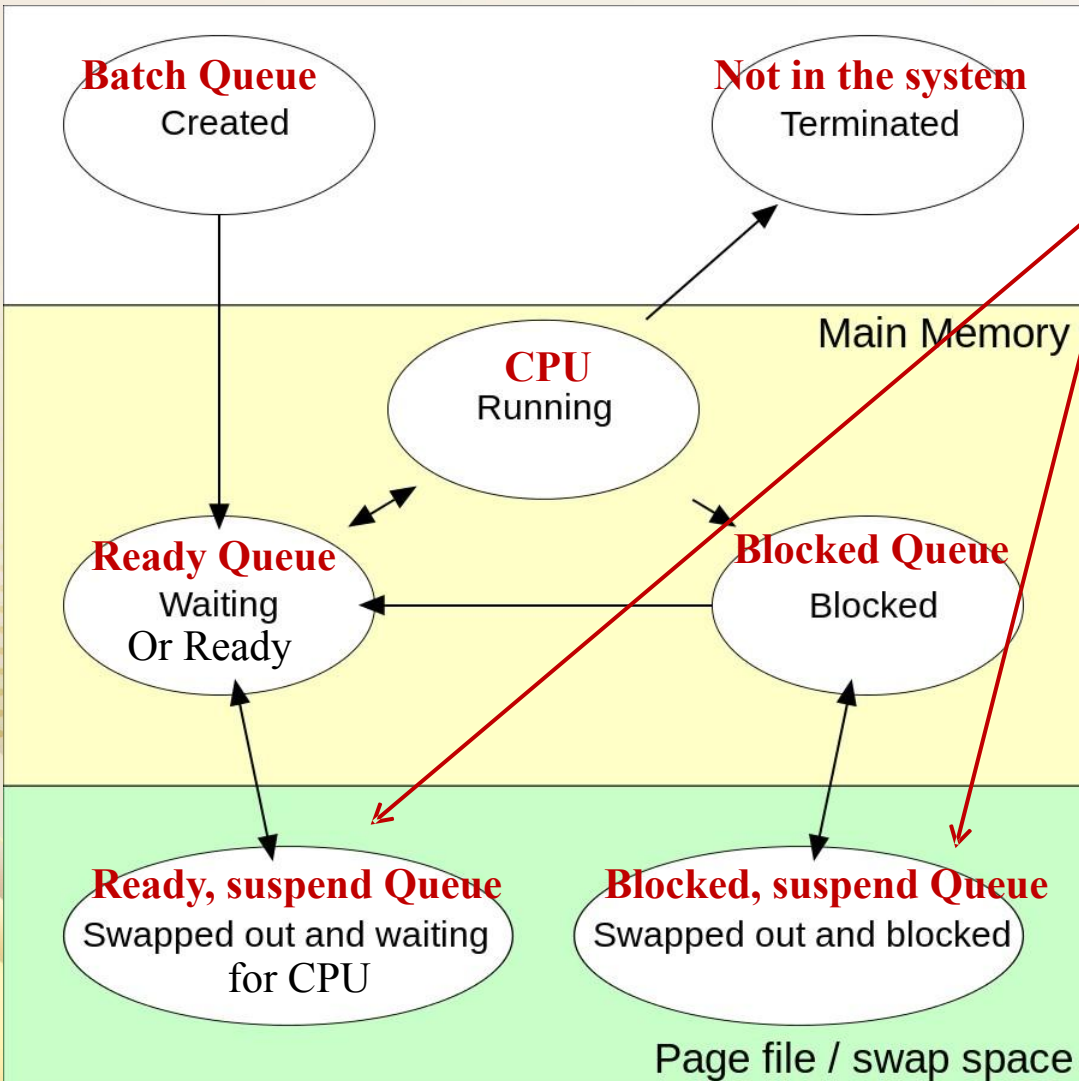
- An operating system must **allocate computer resources** among the potentially **competing requirements** of **multiple processes**.
- In the case of **processor**, the **resource** to be allocated is **execution** time on the processor and the means of allocation is scheduling.
- The **scheduling function** must be designed to satisfy a number of **objectives**, including:
 - Fairness,
 - Lack of starvation of any particular process,
 - Efficient use of processor time, and
 - Low overhead - **Scheduler should do its job within a minimum time.**
- In addition, the scheduling function may need to take into account different levels of priority or real-time deadlines for the start or completion of certain processes.

Quiz 2: Tasks in different states and where they reside

In which queue the tasks in each state reside?

Notes:

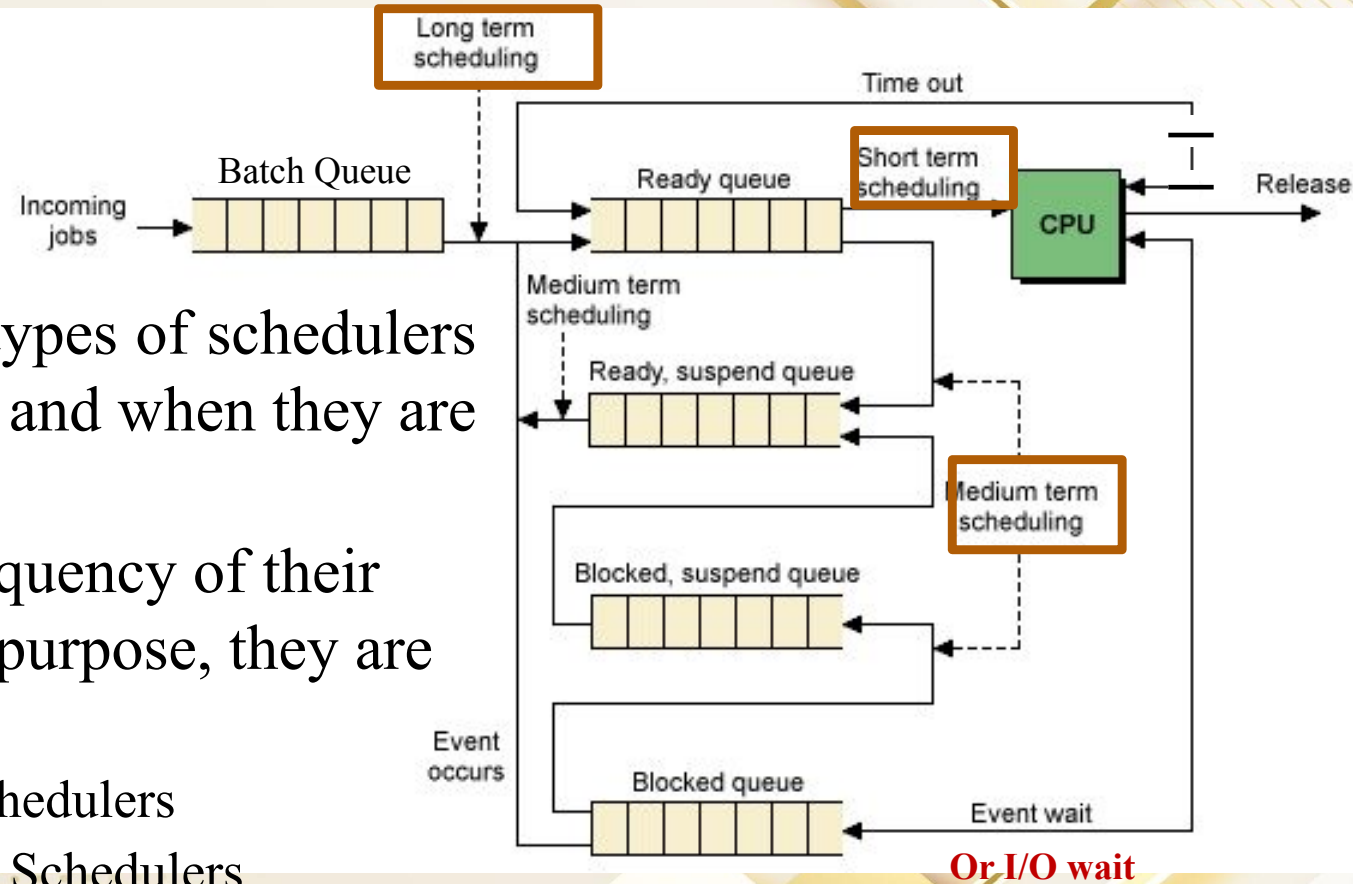
1. Swapped out in **Ready** state.
2. Swapped out in **Blocked** state.
3. Medium-term scheduler **swaps in** or **out** processes to **balance** the no. of tasks in the **Main Memory**.



Types of Schedulers

- There are three types of schedulers based on its role and when they are invoked.
- Based on the frequency of their invocations and purpose, they are categorized as:
 1. Long-Term Schedulers
 2. Medium-Term Schedulers
 3. Short-Term Schedulers

The picture shows the schedulers and the queues they operate on.



Note: The schedulers being considered here are uniprocessor schedulers where single CPU is present.

Long-term Schedulers

Function:

- Selects processes from the **job pool (disk)**.
- Loads them into **main memory**.
- Moves processes from **New** → **Ready** state.

◆ Key Role:

- Controls **degree of multiprogramming**.
- Decides how many processes enter the system.

◆ Characteristics:

- ✓ Runs less frequently
- ✓ Controls system load
- ✓ Present in batch systems

Short-Term Scheduler

Function:

- Selects one process from the **Ready queue**.
- Allocates CPU to it.
- Moves process from **Ready** → **Running**.

♦ Key Role:

- Performs **context switching**.
- Runs very frequently (milliseconds).

♦ Characteristics:

- ✓ Very fast
- ✓ Executes often
- ✓ Core part of multitasking

Medium-Term Scheduler

Function:

- Temporarily removes processes from memory.
- Moves them to disk (suspended state).
- Brings them back later.

♦ Key Role:

- Controls memory usage.
- Manages **suspended processes**.

♦ Characteristics:

- ✓ Performs swapping
- ✓ Improves system performance
- ✓ Used in time-sharing systems

Long-term Schedulers

- If this scheduler is present in the system, it works with batch queue and selects the next batch job to be selected to insert into the Ready queue.
- It admits **new jobs** into Ready queue
- The primary objective is to have a balanced mix of jobs (Processor and I/O intensive).
- First level of throttle on **resource** utilization.
 - Resource here refers to CPU and MM
- Normally invoked when a completed job departs the system.
- Frequency of invocation is generally **much lower**, which is system and workload dependent.
 - This scheduler can incorporate computationally intensive and complex scheduling algorithms.
 - Because this scheduler is going to be invoked infrequently.

Types of Schedulers

Remember only the resident-set of the swapped out processes reside in HDD, whereas the Suspended queues are maintained in OS data structures in MM.

Note: Task and Process are **synonyms**, they could be used interchangeably.

Note: Whereas **Threads** normally mean **light-weight processes**.

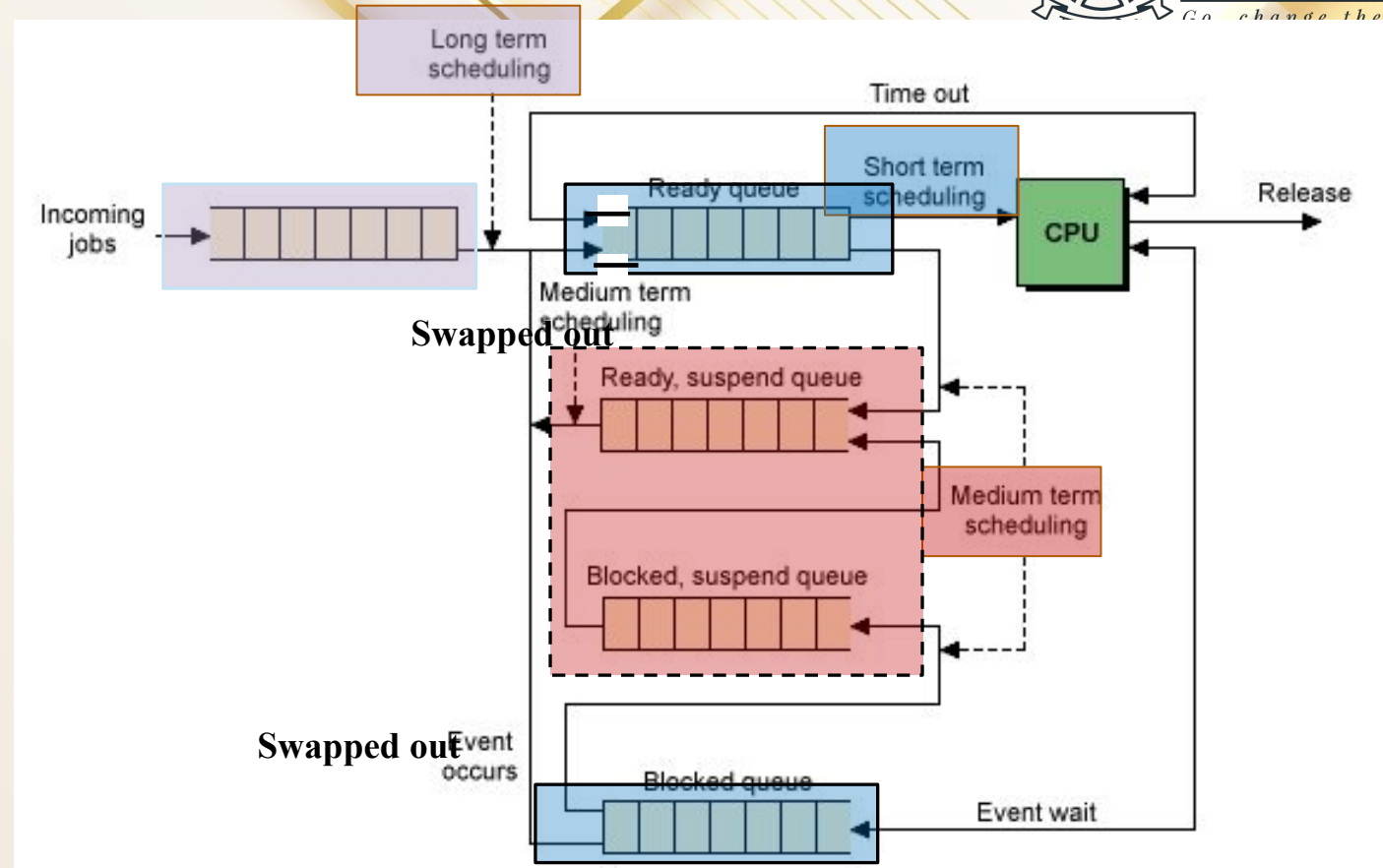
Definitions:

Blocked: Blocked tasks that are in MM

Suspended: Tasks which are **swapped out**. (the tasks could be in Blocked as well as Ready states). Even Ready tasks could be swapped out, and get into Suspend state.

Batch Queue

Swapped in



**In the Main
Memory**

**In secondary
storage**

Medium-term Schedulers

- A running process may become **blocked** due to an I/O request or a system call and cannot proceed until the blocking condition is cleared.
- To free main memory and allow more ready processes, the **medium-term scheduler** may swap blocked or suspended processes to disk.
- When the number of ready processes becomes low, suspended ready processes are swapped back into main memory.
- Once the blocking condition is removed, processes from the **blocked-suspend queue** are swapped in from secondary storage.
- The medium-term scheduler is invoked when processes leave memory or the ready queue size falls below a limit, and it exists only in systems that support swapping.

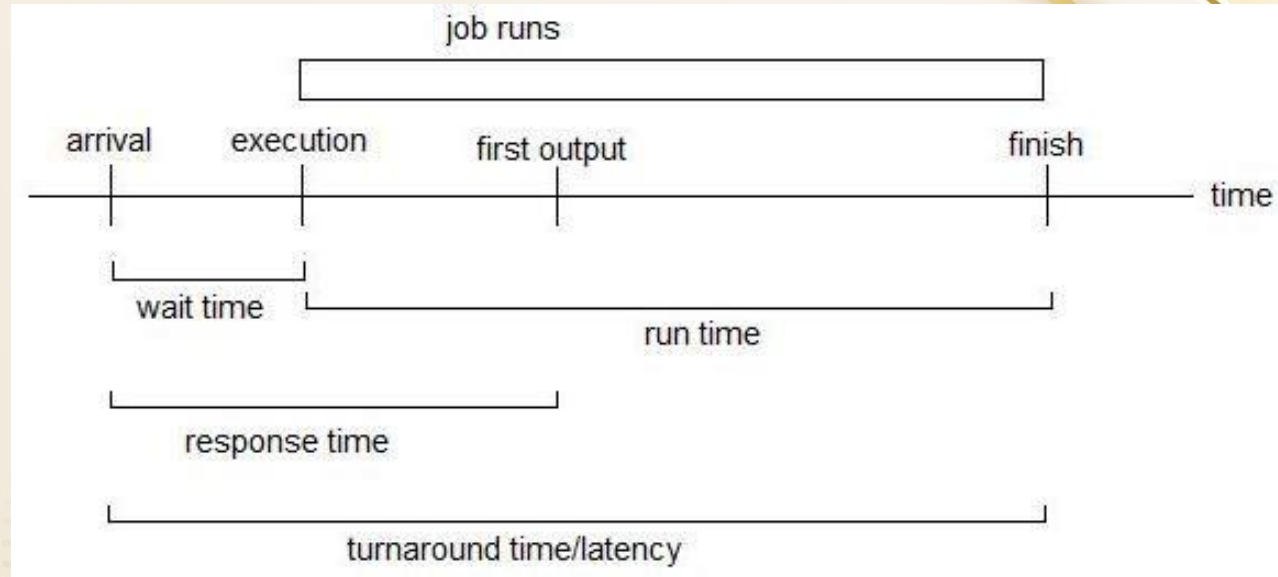
Medium-term Schedulers ...contd.

- This scheduler allocates the CPU from the pool of processes in the Ready queue which are resident in memory.
- It is invoked for each **process switch** to select the next process to be run.
- It is invoked whenever an event (internal or external) causes the global state of the system to change:
 - On time slice (quantum) completion.
 - Interrupts and I/O completion because of which a blocked process becomes Ready.
 - OS system calls get invoked or get completed.
 - Sending or receiving of signals (we will study about this later).

Two types of Scheduling

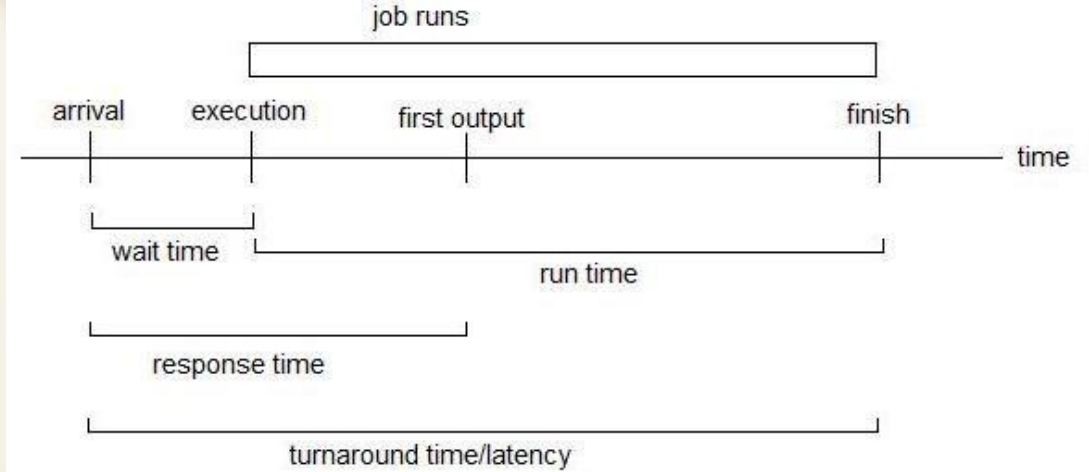
- **Preemptive Scheduling**- Scheduling in which a running process (on CPU) can be interrupted for one of the following reasons:
 - If a higher priority process enters the queue or
 - The time quantum is over for the currently running process, and it is forcibly taken out of CPU
 - In this case, the currently running process is moved to the ready queue, and the process at the head of Ready queue or a higher priority process is made to run on the CPU.
- **Non-preemptive scheduling**- The scheduling in which a running process on the CPU, cannot be interrupted by the OS is called
 - Any other process which enters the Ready queue has to wait until the currently running process finishes its execution on the CPU and voluntarily relinquishes the CPU.
 - There is no fixed time quantum is allocated to the running processes, it is free to occupy the CPU until it relinquishes the CPU on its own.

Scheduler Performance Related Parameters



- Scheduler performance is measured in terms of various timing parameters associated with processes or jobs being scheduled by the Scheduler.
- The goal of the scheduler is to make the tasks to complete their execution at the earliest by utilizing the CPU efficiently.

Scheduling Criteria



- **CPU utilization** – Keep the CPU as busy as possible **productively**.
- **Throughput** – No. of processes that complete their execution per time unit.
- **Turnaround time** or **Latency** – Total amount of time taken to execute and complete a particular process, from the time the process initially became ready.
- **Waiting time** – Total amount of time a process has been waiting in the ready queue.
- **Response time** – Amount of time taken from the time when a job request was submitted (job released or became ready) till the time at which the first response it produced (for interactive jobs).

Quiz 1: Optimization Criteria

- The scheduling criteria are optimization problems.
- Scheduler should either maximize or minimize each of them.
- **Question: Maximize or Minimize?**
 - CPU Utilization: ↑
 - Throughput: ↑
 - Turnaround time: ↓
 - Waiting time: ↓
 - Response time: ↓
- Can all criteria be optimized simultaneously?
- Usually scheduling algorithms try to optimize average values instead of individual values.

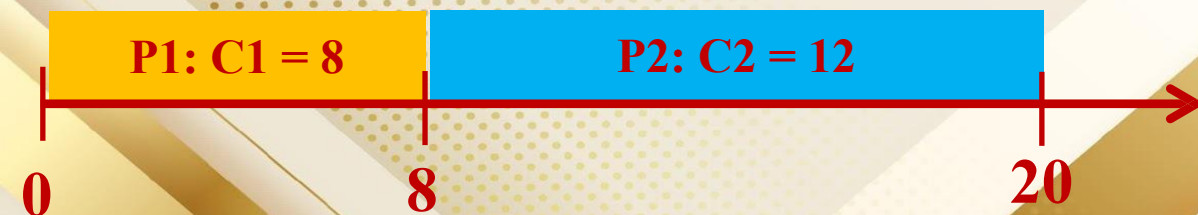
Maximize ↑
Minimize ↓

Example 1: Compute Scheduling Criteria

- Suppose processes **P1** and **P2** arrive at time **zero**.
- **P1: C1 = 8** and **P2: C2 = 12** (computation time for each).
- Consider the time taken for the dispatch of each process is negligible.
- Assume **non-preemptive** scheduler is running.
- Compute the following:
 - Throughput:

$$\frac{\text{No. of Processes completed}}{\text{total time}} = 2/20 = 0.1$$

- CPU utilization: = 100%
- Avg. Turnaround: = $(8 + 20)/2 = 14$
- Avg. Wait time: = $(0 + 8)/2 = 4$



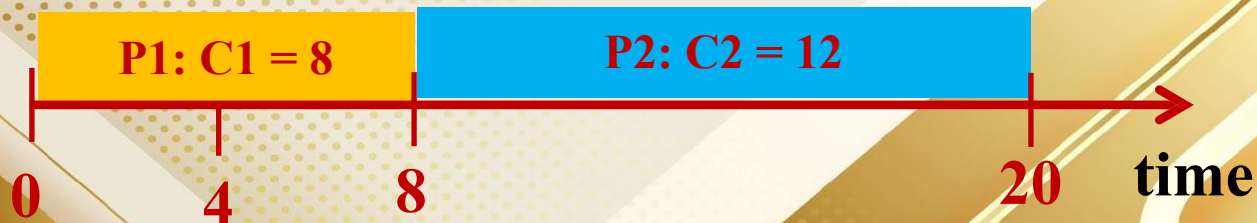
Note 1: Assume time taken by the Scheduler for context switch is negligible,

Note 2: When two processes arrive at the same time, process with lower ID is added to the queue first.

Note 3: CPU utilization is 100% because time taken by the scheduler is negligible.

Example 2: Computing Criteria

- Suppose process **P1** arrives at time **zero** and **P2** at **4**
- **P1: C1 = 8** **P2: C2 = 12** (computation time)
- Consider the time taken for the dispatch of each process is negligible
- Assume **non-preemptive** scheduling
- Compute the following:
 - Throughput: $\frac{\text{No. of Processes completed}}{\text{total time}} = 2/20 = 0.1$
 - CPU utilization: $= 100\%$
 - Avg. Turnaround: $= (8 + 16)/2 = 12$
 - Avg. Wait time: $= (0 + 4)/2 = 2$

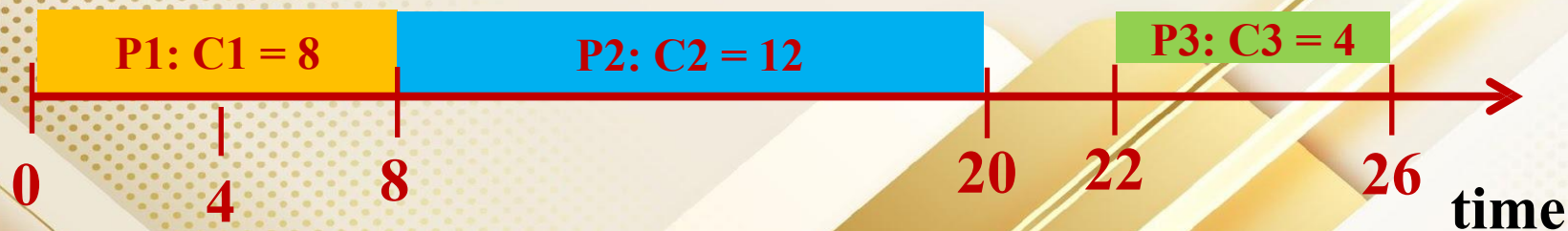


Quiz 2: Computing Criteria

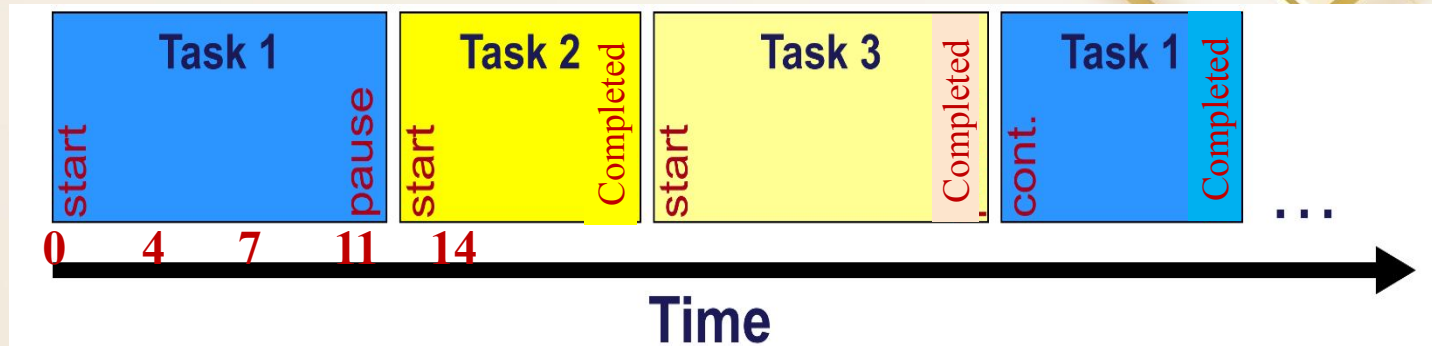
- Suppose process **P1** arrives at time **zero**, **P2** at **4**, **P3** at **22**.
- **P1: C1 = 8, P2: C2 = 12, P3: C3 = 4** (computation time)
- Assume **non-preemptive** scheduling

Compute the following:

- Throughput: $\frac{\text{No. of Processes completed}}{\text{total time}} = 3/26 = 0.115$
- CPU utilization: $= 24/26 = 92.3\%$
- Avg. Turnaround: $= (8 + 16 + 4)/3 = 28/3 = 9.33$
- Avg. Wait time: $= (0 + 4 + 0)/3 = 4/3 = 1.33$



Quiz 3: Choose all the correct options



Note: Assume all the tasks given above get completed. They have nothing to execute beyond what is shown here.

• Choose all the correct statements about the tasks given above:

A. Execution time of Task 3 is the largest of all.

ANS: B and C

B. The task with the shortest execution time is Task 2.

C. The task with the largest execution time is Task 1

D. All the tasks here got completed without any context switches in between

E. Certainly Task 3 should have become ready after Task 1 and Task 2.

Note: Though Task 3 is taken for execution later, nothing can be said about its time of becoming ready until specifically mentioned.

Quiz 4: PCB entries

- Assume **Process 1** is moved from **running state to blocked state**, because of an I/O request then choose the elements in it's PCB which would have **got changed certainly** because of this state change and give a reason for each:

ANS:

- State field changes from Running to Blocked.**
- Program Counter value stored here is the address of the instruction to be executed when this process gets a chance to execute next.**
- Context data will be changed fully.**
- I/O status information gets changed because the state change is from Running to Blocked, because of this process having initiated an I/O request.**
- Accounting information, if it contains for example, I/O devices accessed, it would have got changed certainly.**

Note: Process Identifier will remain the same throughout the life of a process.

Identifier
State
Priority
Program Counter
Memory Pointers
Context Data
I/O Status Information
Accounting Information
⋮

PCB of Process 1

FCFS Scheduler

- **FCFS:** First-Come-First-Served or FIFO (First-In-First-Out).
- This is a **Round-Robin scheduler** with **Quantum** = ∞ .
- The processes are allowed to run until they either complete their execution or they get blocked because of making an I/O call or blocking system calls.
- It is effectively a **non-preemptive scheduler**.
- Processes can get starved if the processes which have arrived earlier, take long time to complete.
- This is not a fair scheduler.
- It favours the processes which are in the system earlier than other processes.

Numerical on FCFS

Table 1

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Find:

- Gantt Chart
- Waiting Time (WT)
- Turnaround Time (TAT)
- Average Waiting Time
- Average Turnaround Time

Step 1:

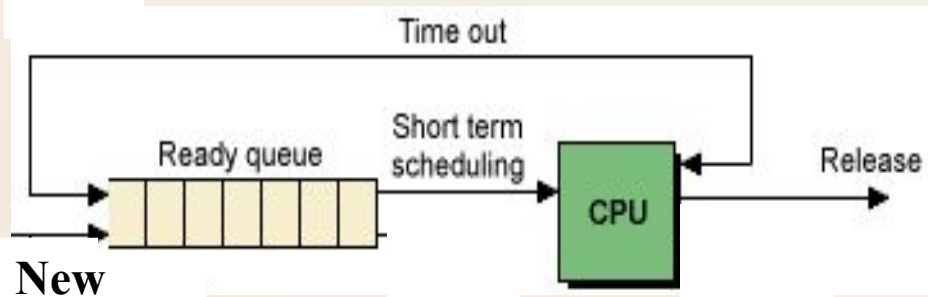
Order of Execution

(FCFS) Processes Execute in order of arrival:

P1->P2->P3->P4

STEP-3 Process.	CT	STEP-3 Process.	CT
P1.	5	P1.	5
P2	8	P2	8
P3	16	P3	16
P4	22	P4	22

Round-Robin (RR) Scheduler



- It picks a **process** at the **head** of the **Ready Q** to be **scheduled** and moves the **currently running process** to the **end** of the **Ready Q**, if it is in Ready state

- **RR** is appropriate for time-sharing environments.
- **Quantum Time (q)**: Amount of time a process gets before being context switched out (also called **time slice**),
 - It is also configurable.
 - A suitable value is chosen based on the type of applications running in the system and the kind of response needed to be provided by the OS.
- **Context switching time** value is **significant** though it is expected to be less than **1% of Quantum time**.
- If **n processes** are running under **round-robin** they will all have an illusion that they have **exclusive access to a CPU** which is running at **1/n times the actual processor speed**

Example 1: RR

- Suppose both **P1** and **P2** processes arrive at time **zero**
- **P1: C1 = 8** and **P2: C2 = 12** (computation time for each)
- Assume **preemptive** scheduling with **Quantum = 4 time units**
- Consider the time taken for the dispatch of each process as

Note 1: When two processes arrive at the same time, process with a lower ID is added to the queue first.

Note 2: Compared to non-preemptive scheduling, both **Avg. Turnaround** and **Avg Wait time** have increased here.

But **preemptive RR scheduler** helps all the **Ready processes** in the system to make **progress**, giving **equal time share** of the **CPU** to all.

No one is starved.

- Compute the following:

◦ Throughput: $= 2/20 = 0.1$

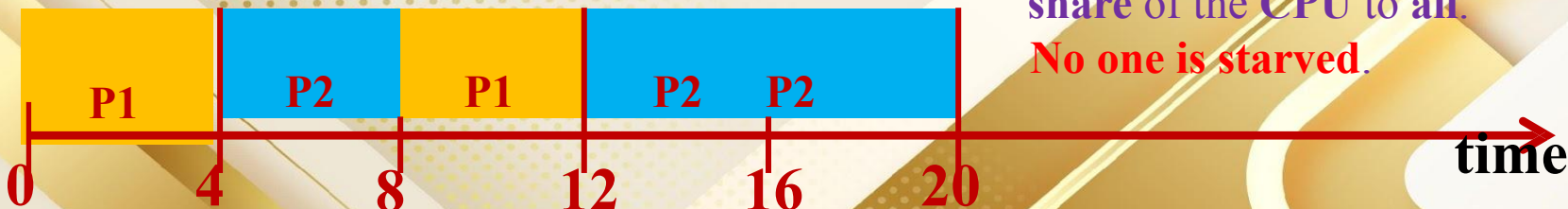
◦ CPU utilization: $= 100\%$

◦ Avg. Wait time:

Non-preemptive case: $= (4 + 8)/2 = 6$

Avg Turnaround time = **14**

Avg Waiting time = **4**



Example 2: RR

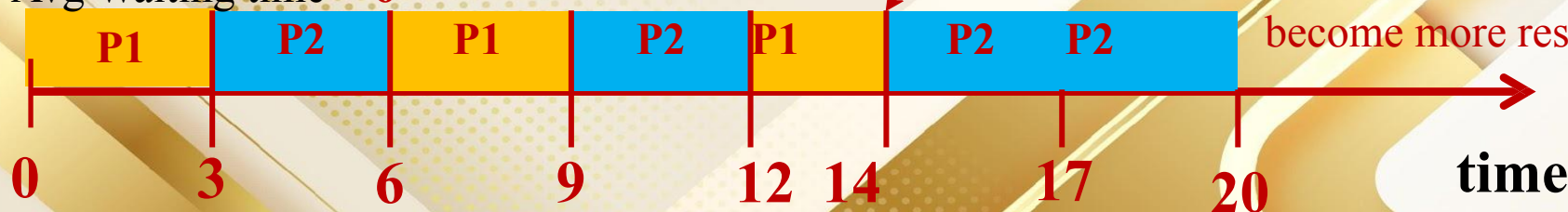
- Suppose both **P1** and **P2** processes arrive at time **zero**
- **P1: C1 = 8** and **P2: C2 = 12** (computation time for each) negligible
- Assume **preemptive** scheduling with **Quantum = 3 time units**
- Consider the time taken for the dispatch of each process as
- Compute the following:

Note: When a process completes, the scheduler kicks in and schedules the other process in the Ready Queue, if there is one, even if the quantum time is not over.

- Throughput: $= 2/20 = 0.1$
- CPU utilization: $= 100\%$
- Avg. Turnaround: $= (14 + 20)/2 = 17$
- Avg. Wait time: $= (6 + 8)/2 = 7$

With Quantum time 4 time units: (Example 1)

Avg Turnaround time = 16
Avg Waiting time = 6



What is the **impact** of **reducing** the **time slice** from 4 to 3 time units?

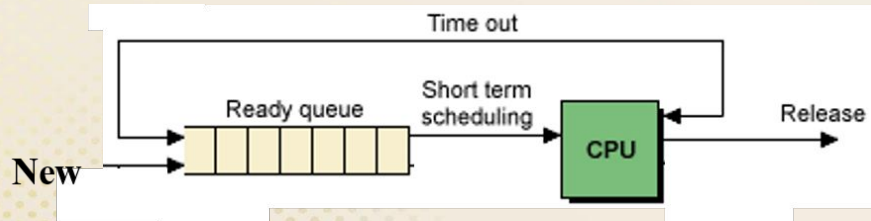
ANS: Since the scheduler is made to run more often here, turnaround and wait time increases, but the tasks become more responsive.

Example 3: Draw the time-line of all the

• Processes in the Table below

In 10 minutes

- **Burst time** is same as **computation time** or **execution time**.
- Assume **RR** scheduler with **time slice = 2 time units**.
- Only draw the **order** in which various **processes get executed** until their
- **completion**. Show the **Ready Q status** too, whenever scheduler gets called.
- Computing scheduler criteria is **Homework**



Note: Assume at a time instant a newly arrived Process enters the Ready queue earlier than the Process evicted out of the CPU at the same time instant.

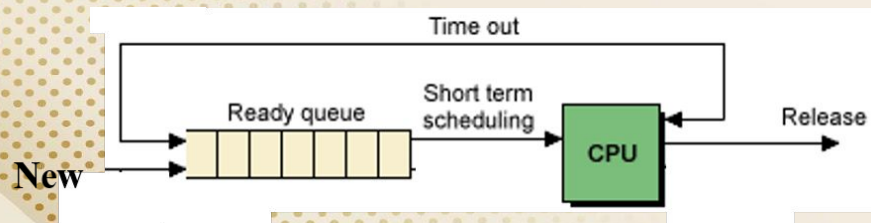
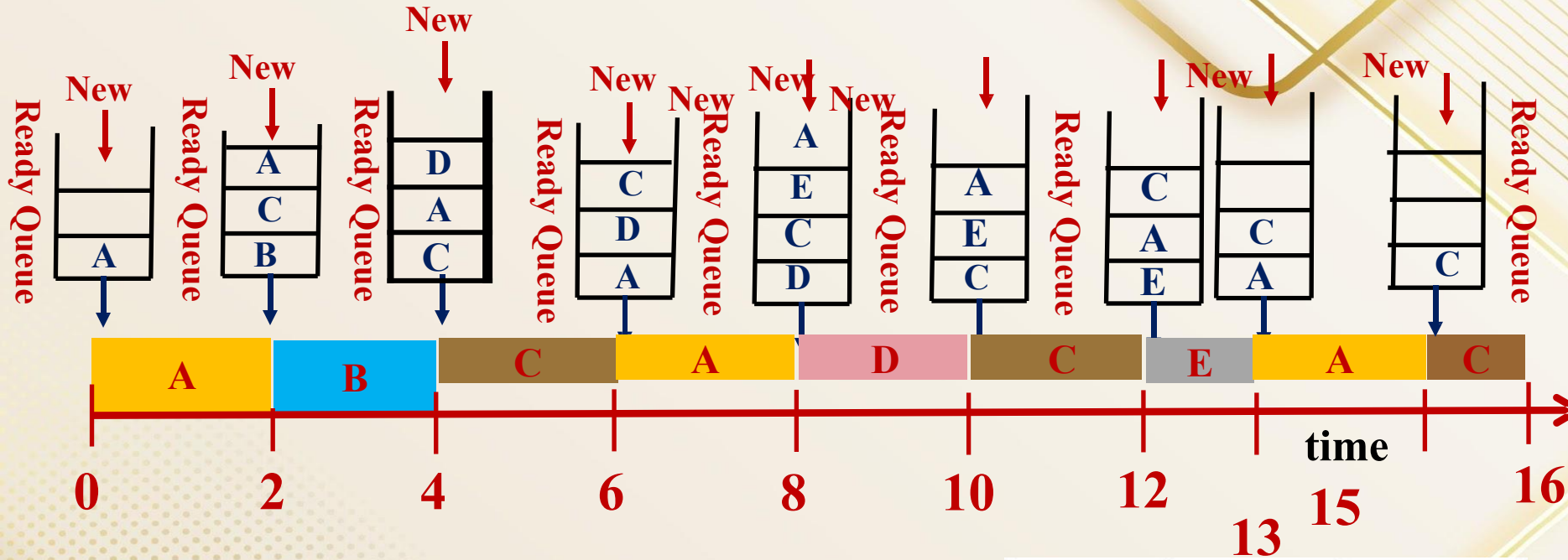
For example, at time 2, Job C will get added to the Ready Q before the process which is currently running on CPU is moved into it.

Job	Arrival Time	Burst Time
A	0	6
B	1	2
C	2	5
D	3	2
E	7	1

**You can do it in
groups too!!!**

**Chocolate for
the First correct
answer!!!**

Example 3: Solution - Updated



Job	Arrival Time	Burst Time
A	0	6
B	1	2
C	2	5
D	3	2
E	7	1

SJF (Shortest Job First) Scheduler



- Out of all available processes, CPU is assigned to the process having the **smallest burst time**.
- In **case of a tie**, it is broken by **FCFS Scheduling**.
- **SJF Scheduling** can be used in both **preemptive** and **non- preemptive mode**.
- **Preemptive mode** of Shortest Job First is called as **Shortest Remaining Time First (SRTF)**.

Problem 1: SJF (non-preemptive)

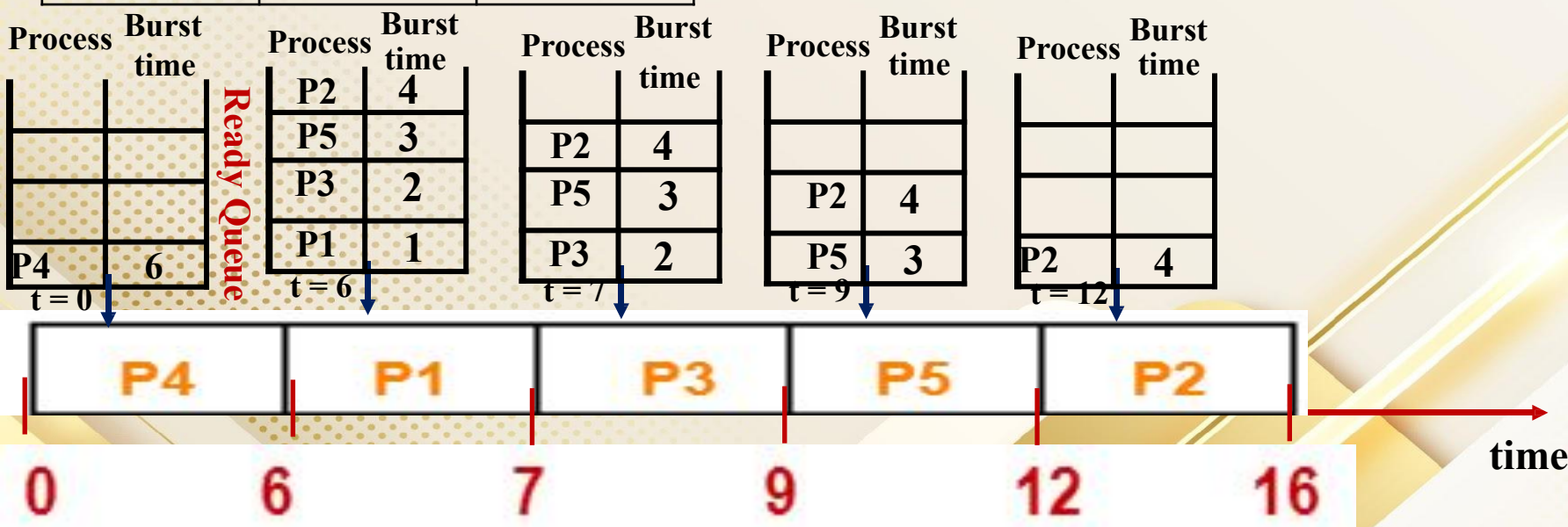
Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Note that, here, the processes are kept in the ready Q not in the order they arrive, but in their order of burst times, with the lowest being at the head of the Ready Q. This is done by the OS on arrival of any new processes.

Ref: Tutorial on SJF

Note: In case of processes With the same burst times, the order of arrival is considered and the earlier one is kept ahead.

Note: Since this is a **non-preemptive scheduler**, once the processes start running, the next scheduling will happen only when the running process relinquishes the CPU on its own.



Picture not to scale

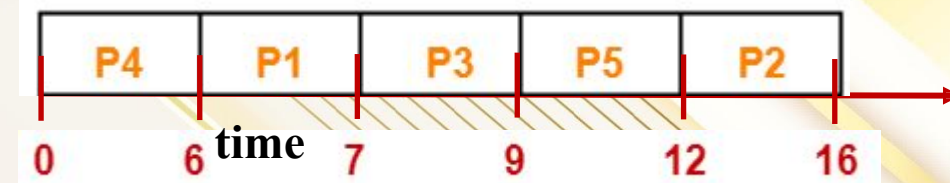
This way of representing processes is called **Gantt chart**

Problem 1: SJF (non-preemptive) contd ..

Find **Turnaround time** and **waiting time** of all the processes

Picture not to scale

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3



Can you give the relationship between them? Execution time, Turnaround time and Waiting time

**Turnaround time = Execution time +
Waiting time**

**Waiting time = Turnaround time –
Execution time**

**Turnaround time = Exit time – arrival
time**

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 1 = 3$
P2	16	$16 - 1 = 15$	$15 - 4 = 11$
P3	9	$9 - 4 = 5$	$5 - 2 = 3$
P4	6	$6 - 0 = 6$	$6 - 6 = 0$
P5	12	$12 - 2 = 10$	$10 - 3 = 7$

Problem 2: SJF (Preemptive) - SRTF

Process ID	Arri val Time	Bu rst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	

Process	Burst time
P4	5
P5	3
P2	3

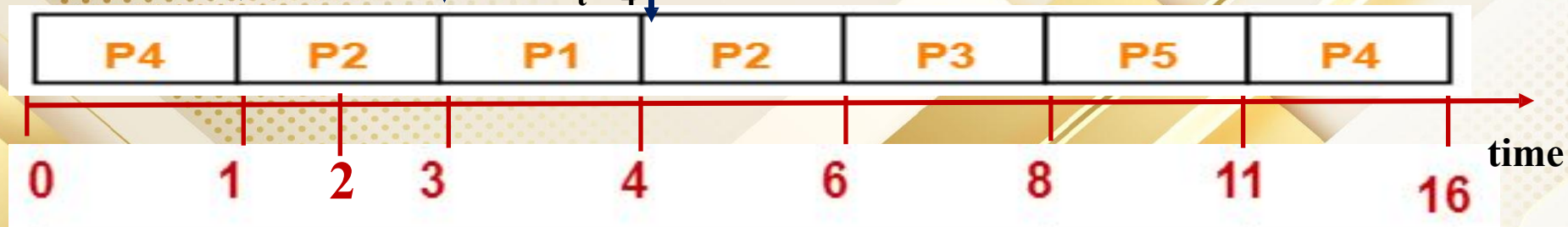
t = 2

SRTF: Shortest Remaining Time First

Note: In case of a tie, it is broken by **FCFS** Scheduling.

Process	Burst time	Process	Burst time	Process	Burst time	Process	Burst time
P4	6	P4	5	P4	5	P4	5
		P2	4	P5	3	P5	3
				P2	2	P3	2
				P1	1	P2	2

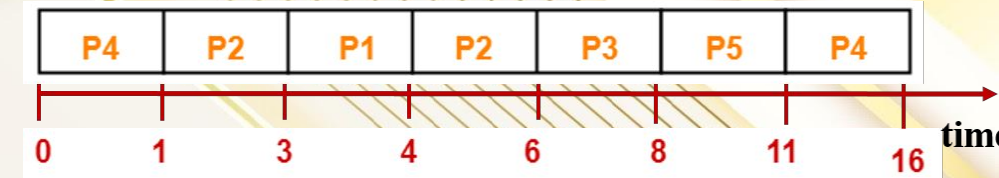
t = 0 t = 1 t = 3 t = 4



Problem 2: SRTF contd ..

Find **Turn around time** and **waiting time** of all the processes

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3



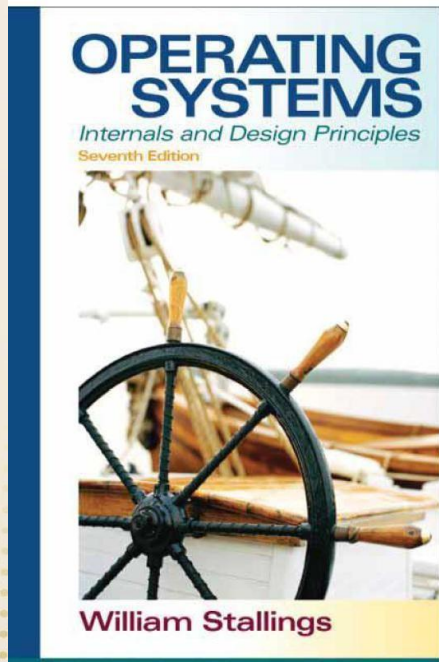
Note:

Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time
Note that if the Burst time is less its waiting time is also less in SRTF.

Process Id	Exit time	Turn Around time	Waiting time
P1	4	$4 - 3 = 1$	$1 - 1 = 0$
P2	6	$6 - 1 = 5$	$5 - 4 = 1$
P3	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	16	$16 - 0 = 16$	$16 - 6 = 10$
P5	11	$11 - 2 = 9$	$9 - 3 = 6$

References (1 to 5)

Ref 1



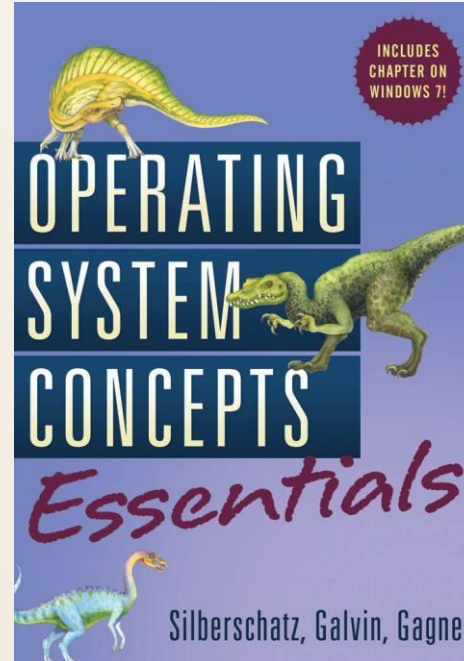
Ref 4

RP2040 A microcontroller by Raspberry Pi

RP2040 Datasheet

A microcontroller
by Raspberry Pi

Ref 2



Ref 5

RP2040 A microcontroller by Raspberry Pi

Raspberry Pi Pico C/C++ SDK

Libraries and tools for
C/C++ development on
RP2040 microcontrollers

Ref 3

